CHAPTER 4: SYSTEM ANALYSIS AND DESIGN

> **Status of this file.** Report-ready content for Chapter 4 of the SBMS final-year project report, produced from a read-only investigation of the `student-beneficiary-system` repository on branch `fix/sbms-completion-work` (commit `d6b5032`), its migrations, tests, evaluation artefacts and project records. Section numbering follows the Chapter 4 pattern of the School exemplar (`CHAPTER_4[1].docx`: 4.1 Introduction, 4.2 Requirement Analysis, 4.3 System Analysis Diagrams, System Design, System Design Diagrams, Database Schema, System Architecture), extended with the data, interface, ML-integration and security subsections required by the project brief.
>
> **Evidence conventions. VERIFIED** = confirmed by reading implementation, configuration, migrations or test output. **PARTIALLY VERIFIED** = some evidence exists but does not establish the full claim. **REQUIRES INPUT** = cannot be confirmed from the repository. **NOT IMPLEMENTED** = the feature does not exist in the inspected code. Every claim that is not fully verified is listed in `DOCUMENTATION_GAP_REGISTER.md`.
>
> **Figures.** Figures are specified, not drawn. Each carries a numbered placeholder, a caption and a specification of its contents, because a diagram that is not derived from the implementation would misrepresent the system.

4.1 Introduction

This chapter presents the analysis and design of the Student Beneficiary Management System (SBMS). The system was built to digitise the registration, identification, verification and bursary-award tracking of student beneficiaries in a Kenyan government-funded welfare programme, and to introduce machine-learning-assisted identification of potential duplicate beneficiary records that a human reviewer then adjudicates. The problem, objectives and development methodology were established in Chapters One to Three. This chapter converts them into an analysable and testable specification: what the system must do, what qualities it must exhibit, what data it must hold, how its parts communicate, and how the delivered artefact was organised.

Three purposes are served. First, the chapter records the requirements against which the implementation is measured in Chapter Five, so that evaluation is anchored to a specification rather than to whatever happened to be built. Second, it documents the analysis models — context, use case, activity, sequence, class and entity-relationship models — that explain how actors, processes and stored data of SBMS relate. Third, it documents the design of the delivered system: its three-tier architecture, module decomposition, access-control model, machine-learning integration, relational schema and physical deployment architecture.

The analysis and design are **derived from the implemented system**, not from an idealised system the project intended to build. Where the implementation departs from the approved proposal, the departure is stated rather than reconciled silently. Two departures are material and recur in this chapter and in Chapter Five:

1. **The duplicate-detection classifier in production is an XGBoost model, not the Random Forest classifier specified in the approved proposal.** The proposal (`Project Proposal- 169613.pdf`, §1.3.1 objective ii and §3.2.3) named a Random Forest classifier over four features. The delivered system uses an `XGBClassifier` inside a scikit-learn pipeline over **six** features (`jaro_winkler`, `dob_match`, `school_match`, `guardian_match`, `upi_match`, `birth_certificate_match`). The Random Forest survives only as a labelled research candidate (`ml-service/model/research/rf_attachment_style_candidate.joblib`) that the application never loads. **VERIFIED** by `ml-service/predict.py:14`, `ml-service/feature_engineering.py:11-18` and `ml-service/evaluation/realistic_pairs_xgboost_training_metrics.json`.
2. **The delivered system exceeds the proposal's scope** in bursary-award verification, payment reconciliation and QR-verifiable award certificates, and **falls short of the scope implied by the proposal's evaluation design** in candidate generation: every new registration is compared against every existing student, with no blocking. The design consequence is analysed in §4.4.4 and evaluated in Chapter Five.

The analytical approach is the one established in Chapter Three: Design Science Research, executed through an iterative Agile cycle, with the machine-learning component following the four-stage pipeline of data acquisition,

preprocessing, training and validation. The analysis artefacts in §4.3 are therefore the artefacts of an iterative build. Neither Structured Systems Analysis and Design Method (SSADM) nor a formal Unified Process was adopted, and no claim is made that the project followed either. Unified Modeling Language (UML) notation is used because the implementation is object-oriented at the module level in both JavaScript and Python, but the governing methodology remains the Agile/DSR process described in Chapter Three.

4.2 Requirement Analysis

4.2.1 Requirements sources and elicitation

Requirements were elicited, as recorded in the approved proposal (§3.3.1), primarily through document analysis of Auditor-General reports, Controller of Budget reports and the NG-CDF legislative framework, because direct access to constituency bursary officers for formal interviews was not available within the academic timeline. That elicitation produced the functional intentions expressed in the proposal. It did **not** produce a numbered requirement register with verification methods, and it did not anticipate the bursary-award reconciliation and certificate functions that were subsequently built.

The register in this section therefore reconciles three sources:

Source	What it contributes	Status
Approved proposal (`Project Proposal-169613.pdf` §1.3, §1.5, §3.3.1)	Original scope: registration with unique identifier and QR code, machine-learning duplicate detection, disbursement recording with audit trail, rule-based eligibility scoring, role-based access control, dashboard, PDF report export, and two named non-functional targets (browser availability; duplicate check under three seconds)	**VERIFIED** as the documented baseline
Implemented system (routers, controllers, services, migrations, frontend routes)	Functions actually delivered, with their access rules, validations and error behaviour	**VERIFIED** by code inspection
Project records (`docs/SBMS_PROJECT_HANDOVER_2026-10.md`, `docs/business-workflow-verification-2026-10-30.md`, `docs/qr-verification-2026-10-30.md`, `docs/bursary-certificates-2026-11-01.md`, `docs/FINAL_TEST_REPORT.md`)	Defects found and fixed, behaviours decided, limits acknowledged	**VERIFIED** as project records; underlying behaviour re-checked against code

Requirement identifiers are assigned here and used unchanged in Chapter Five and in `OBJECTIVE_REQUIREMENT_TEST_TRACEABILITY.md`. Priority follows the MoSCoW convention: **M** (must have), **S** (should have), **C** (could have), **W** (will not have in this release). A requirement is treated as *implemented* only where a route, a service, a database object and a user-facing affordance all exist. Where the interface exists but no executed test covers the behaviour, the requirement is described as *implemented, not fully tested* and is carried into Chapter Five §5.4.12.

4.2.2 Functional requirements

Table 4.1 states the functional requirements of the delivered system. Each is a single testable statement with its actor, priority, rationale and intended verification method. The "Verification" column names the suite or record that establishes the behaviour; where no test exists, the row says so explicitly.

Table 4.1 (part 1 of 3): Authentication, beneficiary management and duplicate review

ID	Requirement statement	Actor(s)	Priority	Rationale / source	Verification
FR-01	The system shall authenticate a user against a stored bcrypt hash and issue a JSON Web Token valid for eight hours.	Administrator, Field Officer	M	Proposal §1.5.1; `controllers/auth.controller.js`	`tests/auth.test.js`; `tests/roleAuthorization.test.js`

ID	Requirement statement	Actor(s)	Priority	Rationale / source	Verification
FR-02	The system shall provide separate sign-in endpoints for the administrator and field-officer roles and refuse an account presented at the wrong endpoint with HTTP 403.	Administrator, Field Officer	M	Separates the two operational roles at the entry point	`tests/auth.test.js`; `tests/roleAuthorization.test.js`
FR-03	The system shall refuse public self-registration with HTTP 410 and create accounts only through an authenticated administrative path.	Administrator	M	Prevents unauthenticated account creation	`tests/auth.test.js`; `tests/roleAuthorization.test.js`
FR-04	The system shall restrict every endpoint to the roles permitted by the access matrix in Table 4.6 and return HTTP 403 otherwise.	All authenticated	M	Duty separation between the two roles	`tests/roleAuthorization.test.js`; `tests/workflow.test.js`
FR-05	The system shall register a beneficiary with full name, date of birth, school identifier, national identifier, primary phone and optional UPI and birth-certificate number, validating each field before persistence.	Field Officer	M	Proposal §1.5.1 (student registration)	`tests/students.test.js`
FR-06	The system shall reject a school identifier that is not one of the five approved schools.	Field Officer	M	Constrains the registry to a controlled school list; API and database both enforce it	`tests/students.test.js`; `students_school_supported_check`
FR-07	On registration the system shall score the new record against every existing student record and create duplicate-review cases for candidates meeting the review policy.	System	M	Proposal objectives ii and iv; the research contribution	`tests/duplicateFlags.test.js`; `tests/workflow.test.js`; `tests/mlPredictor.integration.test.js`
FR-08	The system shall never reject, delete or amend an existing beneficiary record as a result of duplicate screening; a flagged registration shall still create the student record.	System	M	Human oversight of algorithmic signals (proposal §1.4)	`tests/workflow.test.js`; `docs/business-workflow-verification-2026-10-30.md` §4
FR-09	The system shall store, for every duplicate case, the model score, model version, review threshold in force and comparison feature vector.	System, Administrator	M	Auditability of algorithmic evidence	`tests/duplicateFlags.test.js`; migration `20261022_ml_duplicate_provenance.sql`
FR-10	The system shall present duplicate cases to an administrator with identifiers masked, and reveal identifiers only on an explicit, audited request.	Administrator	M	Privacy by default with controlled disclosure	`tests/duplicateFlags.test.js`
FR-11	The system shall record a reviewer decision of `confirmed_duplicate`, `not_duplicate` or `needs_verification` with a mandatory reason, appended to a per-case history.	Administrator	M	Human adjudication replaces automatic decision	`tests/duplicateFlags.test.js`; `tests/workflow.test.js`

Table 4.1 (part 2 of 3): Eligibility, awards and payments

ID	Requirement statement	Actor(s)	Priority	Rationale / source	Verification
FR-12	The system shall compute an eligibility score from five weighted factors and assign one of four eligibility categories.	Field Officer	S	Proposal §1.5.1 (rule-based eligibility scoring)	Role and route covered by `tests/students.test.js`; **no automated test of the scoring arithmetic was found**
FR-13	The system shall allow a field officer to submit a bursary-award application for an existing student with a requested amount.	Field Officer	S	Award-reconciliation extension	`tests/bursaryAwards.controller.test.js`

ID	Requirement statement	Actor(s)	Priority	Rationale / source	Verification
FR-14	The system shall allow a field officer to record a beneficiary verification outcome with an evidence reference, and require an evidence reference for a `verified` outcome.	Field Officer	S	Addresses the missing-documentation problem in Chapter One	`tests/bursaryAwards.controller.test.js`; `tests/workflow.test.js`
FR-15	The system shall permit an administrator to approve or reject an award with a reason, and refuse approval unless the beneficiary is verified and no duplicate case is open or confirmed for that student.	Administrator	M	The central anti-duplication control over payment	`tests/bursaryAwards.controller.test.js`; `tests/workflow.test.js`
FR-16	The system shall record a payment as `issued` against an approved award, refuse payment for an unapproved award, an unverified beneficiary or a student with an unresolved or confirmed duplicate, and refuse a payment exceeding the approved amount.	Administrator, Field Officer	M	Proposal §1.5.1 (benefit-distribution tracking with audit trail)	`tests/disbursements.test.js`; `tests/disbursementReconciliation.controller.test.js`; `tests/workflow.test.js`
FR-17	The system shall record payment clearance separately from issuance and require an evidence reference for clearance.	Administrator, Field Officer	S	Distinguishes instruction to pay from confirmation of settlement	`tests/disbursementReconciliation.controller.test.js`; `tests/workflow.test.js`
FR-18	The system shall record an officer-assisted receipt acknowledgement and classify a shortfall as a `discrepancy` exception rather than overwriting the recorded amount.	Administrator, Field Officer	S	Preserves evidence of a mismatch for follow-up	`tests/workflow.test.js`
FR-19	The system shall report reconciliation exceptions, including unlinked legacy payments, unverified payments, partial payments, awaiting acknowledgement, overdue acknowledgement, failed or cancelled payments and amount discrepancies.	Administrator, Field Officer	S	Makes accountability gaps visible rather than silent	`tests/workflow.test.js`; rules in `services/bursaryRules.js`
FR-20	The system shall generate a unique, cryptographically random QR token per registered student and return a QR image to the registering officer.	System, Field Officer	M	Proposal §1.5.1 (QR-based identification)	`tests/verification.test.js`
FR-21	The system shall expose a public endpoint that reports only whether a student record matching a presented token exists, and nothing else.	Public verifier	M	Identification without disclosure of beneficiary data	`tests/verification.test.js`; `tests/verificationRoute.test.js`

Table 4.1 (part 3 of 3): Certificates, reporting, administration and operations

ID	Requirement statement	Actor(s)	Priority	Rationale / source	Verification
FR-22	The system shall allow an administrator to issue a QR-verifiable certificate for an approved award, and refuse a second certificate for the same award.	Administrator	C	Added deliverable; verifiability of an approval	`tests/bursaryCertificates.test.js`
FR-23	The system shall allow an administrator to revoke a certificate with a reason; public verification shall then report `revoked`.	Administrator	C	Correctness of a document once handed to a holder	`tests/bursaryCertificates.test.js`
FR-24	The system shall report a certificate as `not_current` when the underlying award is no longer approved or its approved amount changed since issue.	System	C	Prevents a document outliving the decision it certifies	`tests/bursaryCertificates.test.js`
FR-25	The system shall present an administrative dashboard with beneficiary, duplicate, award and payment totals, a six-month registration and payment trend, and recent audit activity.	Administrator	S	Proposal §1.5.1 (real-time dashboard)	`tests/roleAuthorization.test.js` (authorisation and synthetic counts); **each counter's arithmetic is not individually tested**

ID	Requirement statement	Actor(s)	Priority	Rationale / source	Verification
FR-26	The system shall export a date-range PDF report containing registration, disbursement, duplicate and accountability summaries.	Administrator	S	Proposal §1.5.1 (reporting with PDF export)	`tests/roleAuthorization.test.js` (range validation); PDF content verified only by code inspection
FR-27	The system shall record an audit entry for every state-changing operation, identifying acting user, action, affected table and record, and contextual details.	System	M	Proposal §1.5.1; accountability is the project's justification	`tests/workflow.test.js`; `tests/duplicateFlags.test.js`
FR-28	The system shall allow an administrator to read and update the duplicate review, high-priority and model cut-offs within 0–1, and audit the change.	Administrator	S	Operational tuning without redeployment	`tests/roleAuthorization.test.js` (denial); `controllers/settings.controller.js` validation
FR-29	The system shall allow an administrator to list users and create administrator or field-officer accounts.	Administrator	S	Administrative provisioning replaces public registration	`tests/roleAuthorization.test.js`
FR-30	The system shall report process liveness and database readiness separately, with readiness failing closed when a required table or the expected database identity is absent.	Operator	S	A live process is not a working system	`test/databaseReadiness.node.js`; `tests/schemaReadiness.test.js`

Three requirements stated in the approved proposal are recorded for completeness, because Chapter Five must evaluate them rather than omit them:

ID	Proposal requirement	Delivered state	Classification
FR-31	Random Forest classifier over four similarity features (proposal §1.3.1 ii, §3.2.2)	An XGBoost classifier over six features is in production; the Random Forest exists only as an unevaluated research candidate	**Implemented with a documented design change** — see §4.4.4 and Chapter Five §5.6
FR-32	Integration with NEMIS or a national identifier verification service	Not implemented; excluded in the proposal (§1.5.1) and never added	**NOT IMPLEMENTED, as scoped**
FR-33	Offline operation and native mobile applications	Not implemented; excluded in the proposal (§1.5.1); the frontend is responsive web only	**NOT IMPLEMENTED, as scoped**

Requirements that the delivered system does **not** satisfy and that the proposal implied must be visible here rather than discovered in Chapter Six:

ID	Gap	Evidence
FR-34	No candidate-blocking or indexing strategy: registration compares the new record with the entire registry, so the number of model calls grows linearly with registry size and the administrator rescan is quadratic	`controllers/students.controller.js:69-73`; `controllers/duplicateFlags.controller.js` scan; `docs/ml-decision-sheet-2026-10-30.md` §6
FR-35	No re-issue path for a student QR token; a student row created without a token (for example by a direct insert) cannot be given one through the API	`docs/qr-verification-2026-10-30.md` §6
FR-36	No token expiry or rotation for student QR codes; a token remains valid for the lifetime of the student row	`docs/qr-verification-2026-10-30.md` §6
FR-37	The system records payments; it does not execute financial transfers. There is no integration with a bank, mobile-money operator or treasury system	`controllers/disbursements.controller.js`; `docs/business-workflow-verification-2026-10-30.md`

4.2.3 Non-functional requirements

Non-functional requirements are stated as measurable constraints wherever the project produced a measurement, and as design constraints where it did not. A requirement without a measurement is labelled as such; it is not presented as achieved. The "Status" column carries the classification defined at the head of this chapter.

Table 4.2 (part 1 of 2): Performance, security, privacy and integrity

ID	Requirement	Measure / constraint	Evidence	Status
NFR-01	Duplicate screening for one registration shall complete within three seconds	≤ 3 000 ms end-to-end for a 50-candidate registration	`backend/tests/performance.test.js`; 22.67 ms measured for 50 pairs (2026-10-30 record)	**PARTIALLY VERIFIED**: satisfied at 50 candidates on the development machine; no measurement at registry scale
NFR-02	Password storage shall resist offline attack	bcrypt with cost factor 12	`controllers/auth.controller.js:26`	**VERIFIED** as implementation; cost factor not benchmarked on the target host
NFR-03	Session credentials shall be signed and time-bounded	HMAC-signed JWT, `expiresIn: '8h'`, secret ≥ 32 random bytes, placeholder rejected at start-up	`controllers/auth.controller.js:55-59`; `config/jwtSecret.js`; `test/jwtSecret.validation.node.cjs`	**VERIFIED**
NFR-04	Authorisation shall not trust a client-supplied claim	Role re-read from the `users` row on every authenticated request	`middleware/auth.js:20-24`; `tests/roleAuthorization.test.js`	**VERIFIED**
NFR-05	Beneficiary identifiers shall not be exposed by default in the review queue	`guardian_id`, `upi`, `birth_certificate_number`, `national_id_number` masked to the last three characters; separate audited reveal	`controllers/duplicateFlags.controller.js:37-42, 106-107`; `tests/duplicateFlags.test.js`	**VERIFIED**
NFR-06	Public verification shall disclose no beneficiary data	Student endpoint returns `{found: boolean}` only; certificate endpoint returns authenticity facts only	`controllers/verification.controller.js`; `controllers/certificates.controller.js:53-89`; `tests/verification.test.js`; `tests/bursaryCertificates.test.js`	**VERIFIED**
NFR-07	Public endpoints shall be rate limited	10 requests per client IP per 15 minutes (student verification); 20 per 15 minutes (certificate); separate budgets	`middleware/verificationRateLimit.js`; `certificateRateLimit.js`; `rateLimitWindow.js`	**VERIFIED** in-process only; counters reset on restart and are not shared between instances
NFR-08	Sign-in shall be rate limited	10 attempts per client IP per 15 minutes	`middleware/loginRateLimit.js`; `tests/loginRateLimit.test.js`	**VERIFIED**
NFR-09	Request bodies shall be bounded	JSON limit 100 kB; oversize returns a safe client error	`index.js:64`; `tests/securityMiddleware.test.js`	**VERIFIED**
NFR-10	Errors shall not leak internals	5xx responses return "Internal server error"; logs carry no stack, message or credential	`middleware/errorHandler.js:59-78`; `tests/securityMiddleware.test.js`	**VERIFIED**
NFR-11	Browser origins shall be restricted by allowlist	Unlisted origin rejected before routing; malformed configuration stops start-up	`config/cors.js`; `index.js:40-47`; `tests/securityMiddleware.test.js`	**VERIFIED**
NFR-12	Data integrity shall be enforced by the database, not only the application	`CHECK` constraints for roles, school list, award decision fields, verification fields, payment and receipt status, clearance and acknowledgement completeness, certificate status and token format; foreign keys throughout	`backend/src/db/schema.sql`; migrations `20261019_*`, `20261023_*`, `20261031_*`	**VERIFIED**; the key constraints are exercised by `tests/workflow.test.js` and `tests/bursaryCertificates.test.js`

ID	Requirement	Measure / constraint	Evidence	Status
NFR-13	Concurrent registrations shall not interleave their duplicate comparisons	Transaction plus `pg_advisory_xact_lock(hashtext('sbms.student.registration'))`; candidates selected `FOR UPDATE`	`controllers/students.controller.js:67-73`; the same lock guards award decisions and payment creation	**PARTIALLY VERIFIED**: the lock exists and is used; **no concurrency test exists**

Table 4.2 (part 2 of 2): Usability, maintainability, operability, oversight and recovery

ID	Requirement	Measure / constraint	Evidence	Status
NFR-14	The interface shall be usable by non-technical officers on varied devices	Responsive layout; role-specific navigation; plain-language screening messages for officers; single-line status pages for public verifiers	`components/AppLayout.jsx`; `pages/StudentRegistration.jsx:7-22`; `pages/Verification.jsx`	**PARTIALLY VERIFIED**: design intent implemented and browser-checked; **no usability study or acceptance session with real officers has been conducted**
NFR-15	The system shall be maintainable and its behaviour reproducible	One feature-schema module shared by training and serving; ordered migration files; Jest, native `node --test` and Python `unittest` suites; version-pinned requirements	`ml-service/feature_engineering.py`; `backend/src/db/migrations/`; `ml-service/requirements.txt`; `docs/FINAL_TEST_REPORT.md`	**VERIFIED**
NFR-16	The system shall remain operable within a zero-cost student cloud budget	Containerised API with `minReplicas 0`; free App Service frontend tier; PostgreSQL stopped between demonstrations; no log-analytics workspace	`render.yaml`; `Dockerfile`; `docs/azure-deployment-log-2026-10-31.md` §1, §3–§4	**VERIFIED** as deployed configuration; the cost model is an owner decision, not a property of the code
NFR-17	A human shall make every duplicate and payment decision	Model scores create review cases only; approval, disbursement and certificate issue are role-gated human actions	`controllers/duplicateFlags.controller.js:452-503`; `controllers/bursaryAwards.controller.js:151-209`	**VERIFIED**
NFR-18	The system shall recover from database loss with verified evidence	Dump, restore into an independent database, readiness and schema/row-digest comparison rehearsed	`docs/database-recovery-rehearsal-2026-10-31.md`	**PARTIALLY VERIFIED**: rehearsal on a local disposable database; **no scheduled backup job or retention policy exists in the deployment**
NFR-19	The system shall be compatible with ordinary modern browsers	Static single-page application over HTTPS; no browser-specific API beyond `fetch`, `history.replaceState` and React Router hash handling	`frontend/src/**`; `frontend/public/web.config`	**PARTIALLY VERIFIED**: exercised in Chrome only; **Firefox, Safari and mobile-browser compatibility are not recorded**
NFR-20	The system shall degrade predictably when the ML predictor is unavailable	Registration aborts with HTTP 503 rather than registering an unscreened beneficiary	`controllers/students.controller.js:74-96`; `services/duplicateCheck.js:284-300`	**VERIFIED** by code inspection and the 503 assertion path in `tests/students.test.js`
NFR-21	Audit records shall be attributable and complete for state-changing actions	Every controller writes an `audit_logs` row with actor, action, table, record and JSON details	`services/auditLog.js`; `tests/workflow.test.js` (ten-action sequence)	**VERIFIED** for the workflows tested; **append-only by convention only — no database-level immutability or retention policy exists**

The following requirements were **not** met, and are stated here so that Chapter Five does not have to discover them:

ID	Requirement not met	Reason and evidence
NFR-22	Load-tested capacity for a constituency-scale registry	No hosted load test has been run; the only performance evidence is one local test at 50 candidates (`docs/FINAL_TEST_REPORT.md` §8)
NFR-23	Thresholds validated against adjudicated beneficiary data	No adjudicated label set exists; decisions D1–D5 remain unanswered (`docs/ml-evaluation-policy.md`)
NFR-24	Independent confirmation of receipt by a school	Acknowledgement is officer-assisted by design and is labelled as such in the interface and in the database (`acknowledgement_method = 'officer_assisted'`)

4.2.4 Data requirements

The data requirements describe what the system stores, where the data originates, how it is validated, how it is protected, and what is deliberately not stored.

Data collected. Four categories of data are held.

1. *Operational identity data for beneficiaries* (`students`): full name, date of birth, school identifier, guardian identifier, UPI, birth-certificate number, national identifier, primary phone, QR token, eligibility score and category, synthetic-data marker, and timestamps.
2. *Account data for staff* (`users`): full name, e-mail, bcrypt password hash, role (`admin` or `field_officer`), active flag, creation timestamp.
3. *Decision and accountability data*: duplicate cases and their review history (`duplicate_flags`, `duplicate_review_events`), bursary awards and their verification history (`bursary_awards`, `bursary_award_verification_events`), payments and their reconciliation events (`disbursements`, `disbursement_reconciliation_events`), issued certificates (`bursary_certificates`), audit records (`audit_logs`) and generated-report metadata (`reports`).
4. *A legacy table* (`beneficiaries`) retained because `/api/beneficiaries` is still mounted and the readiness check requires it. Its `scholarship_amount` column has no approved mapping to any award or payment amount, and the newer student model and the legacy beneficiary model are **not reconciled** (`docs/SBMS_PROJECT_HANDOVER_2026-10.md` §1.2).

Data sources and ownership. All beneficiary data in the delivered system is entered by a field officer through the registration form, or generated by the system (identifiers, timestamps, QR tokens, model scores). The repository contains **no real beneficiary records**: the demonstration registry is synthetic, and synthetic rows are explicitly marked `is_synthetic = TRUE` and counted separately on the dashboard (`controllers/adminDashboard.controller.js:26-32`). Data ownership therefore sits with the implementing organisation, and no claim is made that personal data of real learners was processed.

Validation requirements. Validation is layered so that a client cannot bypass it.

Layer	Mechanism	Example
Transport	JSON body limit 100 kB; exact-shape checks on public endpoints (exactly one key, `token`)	`index.js:64`; `controllers/verification.controller.js:7-15`
Schema	Zod schemas per endpoint: trimmed lengths, e-mail format, ISO date, `+254[17]\d{8}` phone, enum of approved schools, UUID identifiers, positive monetary amounts bounded at 9 999 999 999.99	`controllers/students.controller.js:15-24`; `controllers/bursaryAwards.controller.js:5-26`; `controllers/disbursements.controller.js:9-38`
Domain rule	Cross-field refinements: a `verified` outcome requires an evidence reference, clearance requires evidence, a failed or cancelled payment requires a reason of at least five characters	`controllers/bursaryAwards.controller.js:14-17`; `controllers/disbursements.controller.js:21-27`
Database	`CHECK` constraints and foreign keys that restate the domain rules	`schema.sql`; migration `20261023_bursary_award_reconciliation.sql:21-31, 70-96`

Normalisation is applied before comparison and before storage so that duplicate detection is not defeated by formatting: names collapse internal whitespace, identifiers are stripped to alphanumerics and upper-cased, and Kenyan phone numbers are normalised to `+254` form (`services/studentRecordFormat.js`; `ml-service/feature_engineering.py`).

Data quality requirements. Three quality rules are enforced by design rather than by inspection.

1. *A missing identifier is never a match.* The feature builder treats empty strings, `NA`, `NULL`, `UNKNOWN`, `NOT PROVIDED`, `NOT AVAILABLE` and runs of zeros as missing, and returns `0.0` for similarity instead of comparing them (`feature_engineering.py:21-33, 81-86`).
2. *A date is compared as a calendar date.* PostgreSQL `DATE` columns arrive at the Node layer as a JavaScript `Date` at local midnight; the boundary to the predictor normalises them to `YYYY-MM-DD` so a stored date and a freshly submitted date are never a day apart (`mlDuplicateReview.js:61-82`; `tests/predictorDateNormalization.test.js`).
3. *An identifier pair is compared once.* A unique index over the unordered student pair (`LEAST(student_id, matched_student_id), GREATEST(student_id, matched_student_id)`) prevents the same pair creating two cases (`schema.sql:95-96`).

Data storage and relationships. Data is stored in one PostgreSQL database reached through a connection pool (`config/db.js`). The relationships are documented in §4.6; in summary: one `users` row creates many `students`; every `student` may be party to many `duplicate_flags`; one `student` may hold many `bursary_awards`; each award may have many `disbursements` and at most one `bursary_certificates` row; and every state-changing action appends to `audit_logs`.

Data protection and access restrictions. Protection is enforced at four levels: TLS for the public HTTPS origins through which all traffic flows; the CORS allowlist and security headers (`X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, `Permissions-Policy: microphone=(), geolocation=()`) on every response (`index.js:33-39`); role-based authorisation on every non-public route; and field-level masking of identifiers in the review queue with an audited reveal. Secrets (database URL, JWT secret, bootstrap passwords) are supplied through environment variables or the hosting platform's secret store, are excluded from the container image by `.dockerignore`, and are never rendered in the interface or written to logs.

Duplicate and missing data handling. Duplicates are deliberately **not** prevented by a unique constraint on the identifier columns: `students.upi` is nullable and its earlier uniqueness constraint was explicitly dropped (`schema.sql:47-49, 57`), because the project's premise is that near-duplicates cannot be detected by exact matching and must instead be routed to human review. Missing values are handled rather than rejected: date of birth, school, guardian identifier, UPI and birth-certificate number are all optional, and a student may be registered with only a name and a national identifier, in which case screening is necessarily weaker and the audit record shows how many features were unavailable.

Machine-learning data. The production model was trained on a generated corpus of 12 000 labelled record pairs (6 000 duplicate groups, balanced 6 000/6 000), drawn from a simulator that corrupts names (misspelling, reordering, truncation), introduces date-of-birth errors and school transfers, changes guardians, reissues identifiers, and adds deliberate confusers — siblings, twins, common names within one school, and reused identifiers on non-duplicates (`ml-service/evaluation/realistic_pairs_xgboost_training_metrics.json`, `generator_scenarios`). Training and evaluation were separated by identity: an 80/20 grouped internal split, then an independently generated holdout of 12 000 pairs from new latent identities and a separate random seed. The holdout shares no persons with the training rows, but it does share the generator, and the metrics report states this in its `limitations` block. The repository contains **no** adjudicated real-world label set: `ml-service/evaluation/reviewed_pairs_template.csv` is header-only (`docs/ml-decision-sheet-2026-10-30.md` §7, D5).

Feature extraction. Six features are computed per pair, in a fixed order, by one module used for both training and serving (`feature_engineering.py:11-18`):

Feature	Type	Derivation
`jaro_winkler`	continuous, 0–1	Maximum of Jaro–Winkler similarity on the concatenated name tokens and on the alphabetically sorted tokens, so reversed name order still scores highly
`dob_match`	0 / 0.5 / 0.8 / 1	1 for an identical calendar date, 0.8 for one day apart, 0.5 for up to seven days, 0 otherwise; five date formats accepted, ambiguous US ordering deliberately not assumed
`school_match`	binary	Exact match of normalised school identifiers
`guardian_match`	continuous, 0–1	Exact identifier match after stripping shared prefixes (`G`, `UPI`, `BC`, …), otherwise Jaro–Winkler similarity
`upi_match`	continuous, 0–1	As `guardian_match`, for the UPI
`birth_certificate_match`	continuous, 0–1	As `guardian_match`, for the birth-certificate number

The national identifier is **not** a model feature. It is used outside the model as a separate human-review signal (`services/mlDuplicateReview.js:33-39`), together with exact UPI and birth-certificate agreement, because an exact identifier agreement is evidence a reviewer can check against a source document whereas a similarity score is not.

Retention, backup and recovery. No retention policy, archival job or scheduled backup is implemented in the application or in the deployment configuration. Backup and restore were rehearsed manually and the evidence is recorded in `docs/database-recovery-rehearsal-2026-10-31.md`; the hosted database is a PostgreSQL Flexible Server stopped between demonstrations, whose storage persists while compute is stopped. The absence of an automated backup schedule is a documented limitation carried into Chapter Six.

4.2.5 Interface and integration requirements

The system has five interfaces. Each is described by its protocol, message format, authentication rule and error behaviour, because the evaluation in Chapter Five depends on those contracts being stated.

Table 4.3: Interfaces and integration points

Interface	Parties	Protocol and format	Authentication	Error behaviour
I-1 Browser to API	React SPA → Express API	HTTPS, JSON over `axios`; `Authorization: Bearer <JWT>` injected by a request interceptor (`frontend/src/services/api.js`)	JWT with 8-hour expiry; invalid or expired token yields 401 and the SPA returns the user to sign-in	4xx with `{message}`; 5xx with a generic message; refused origin yields 403
I-2 API to PostgreSQL	Express → `pg` pool	TCP with TLS to the managed server; parameterised SQL only	Database credentials from the environment	Errors logged without SQL text or parameters; generic 500 returned
I-3 API to ML predictor	Node child process → `predict.py --server`	Line-delimited JSON over process standard input/output; request `{pairs:[{first,second}]}`, response `{model_version, results:[{score, features}]}`	Local process boundary only; never exposed on a network socket	Predictor exit, 30 000 ms timeout or malformed payload raises `DUPLICATE_PREDICTOR_FAILED`, which registration converts to HTTP 503
I-4 Student QR verification	QR → public page → API	QR encodes `…/verify#token=<64 hex>`; the page posts `{token}` to `POST /api/verify`	Public; rate limited to 10 requests per IP per 15 minutes	Malformed, unknown and refused tokens all return `{found:false}`; lookup failure returns 503 `{error:'temporarily_unavailable'}`; rate refusal returns 429
I-5 Certificate QR verification	QR → public page → API	QR encodes `…/verify-certificate#token=<64 hex>`; the page posts `{token}` to `POST /api/certificates/verify`	Public; separate budget of 20 requests per IP per 15 minutes	`not_recognised` for any malformed, unknown or manipulated token; `revoked`; `not_current`; `verified`; 503 on lookup failure

Two further integration points are **absent**, recorded so that the report does not imply more than was built. PDF rendering is internal — `pdfkit` writes the report to a `reports/` directory and renders the certificate PDF per request in memory — and there is no external document service. There is also no external government or financial

integration: NEMIS, the national identifier verification service and any bank or mobile-money transfer interface are absent by scope (FR-32, FR-37).

The QR design deserves separate comment because it carries most of the project's privacy argument. Both token families are 32 random bytes rendered as 64 lowercase hexadecimal characters and are placed in the **URL fragment**, not the query string. A fragment is never transmitted to a web server, never written to an access log and never forwarded as a `Referer` (`config/publicAppUrl.js:164-177`). The public pages read the fragment, remove it from the visible address with `history.replaceState`, and post it to the API (`frontend/src/hooks/useHashToken.js`; `pages/Verification.jsx`; `pages/CertificateVerification.jsx`). `PUBLIC_APP_URL` is validated at start-up and again at issue time, and a non-public origin — including a LAN address, a bare hostname or an `example.*` domain — is refused, so a QR code can never be generated that a phone on a mobile network cannot reach (`config/publicAppUrl.js:119-162`; `docs/qr-verification-2026-10-30.md` §5).

4.3 System Analysis Diagrams

This section presents the analysis models. Each figure is specified rather than embedded: the placeholder states exactly what the diagram must contain, so that the inserted diagram is derived from the implementation and not from an idealised description of it. The notation is UML 2.x for the use case, activity, sequence and class models and Chen notation for the entity-relationship model. Terminology is fixed across all figures and matches the database and the code: *Administrator* (`admin`), *Field Officer* (`field_officer`), *Public verifier*, *Duplicate predictor* (the Python process), and the data objects `students`, `duplicate_flags`, `duplicate_review_events`, `bursary_awards`, `bursary_award_verification_events`, `disbursements`, `disbursement_reconciliation_events`, `bursary_certificates`, `audit_logs`, `reports`.

4.3.1 System context diagram

A context diagram is required because the system boundary matters: several capabilities that a reader might assume belong to a beneficiary management system are deliberately outside it. The context diagram shows SBMS as a single process and the external entities that exchange data with it. It is **not** a level-0 data flow diagram: it shows no internal processes and no data stores, and its purpose is to fix the boundary.

[Insert Figure 4.1: System context diagram of SBMS here]

Figure 4.1: System context diagram. The figure must show the system as one rounded process labelled *SBMS (React SPA, Express API, PostgreSQL, Python duplicate predictor)* at the centre, with the external entities and the direction of each flow:

External entity	Flows into SBMS	Flows out of SBMS
Field Officer	sign-in credentials; beneficiary registration data; award applications; verification outcome and evidence reference; payment and clearance entries; receipt acknowledgement	JWT session; student lists; QR image for the registered student; screening outcome in plain language; eligibility assessment; award and payment status
Administrator	sign-in credentials; award decisions; duplicate-case decisions; identifier-reveal requests; threshold changes; certificate issue and revocation instructions	duplicate queue with masked identifiers; dashboard aggregates; audit log; PDF reports; certificate PDF and QR
Public verifier (school, guardian, awarding office)	student QR token or certificate token scanned from a printed document	`found` / `not found`; certificate status (`verified`, `revoked`, `not_current`, `not_recognised`)
Duplicate predictor (shown as an external collaborator because it is a separate process)	six ordered feature values per candidate pair	model score and model version per pair
Hosting platform (Container Apps, App Service, PostgreSQL Flexible Server)	runtime, TLS termination, database service	container image, static frontend bundle, database state

The diagram must **not** show NEMIS, a bank, a mobile-money operator, an SMS gateway or a national identifier verification service, because none is integrated. It must also not show the system automatically rejecting, deleting or mutating a beneficiary record, because no such flow exists.

4.3.2 Use case diagram

[Insert Figure 4.2: Use case diagram of SBMS here]

Figure 4.2: Use case diagram. Three actors sit outside the system boundary: **Field Officer**, **Administrator** and **Public verifier**. *Duplicate predictor* is an internal collaborating actor, connected only to the registration and rescan use cases. The use cases, with their actor associations, are:

Use case	Field Officer	Administrator	Public verifier	Included / extending behaviour
UC-01 Sign in (role-specific page)	✓	✓	—	«include» verify credentials; «extend» refuse wrong-role account
UC-02 Register beneficiary	✓	—	—	«include» UC-03
UC-03 Screen for duplicates	—	—	—	«include» score pairs; «include» select review cases per aggregation policy
UC-04 View registered beneficiaries	✓	—	—	—
UC-05 Assess eligibility	✓	—	—	—
UC-06 Submit bursary award	✓	—	—	«include» require school on record
UC-07 Record beneficiary verification	✓	—	—	«include» require evidence reference when verified
UC-08 Review duplicate queue	—	✓	—	«include» UC-09
UC-09 Reveal identifiers (audited)	—	✓	—	—
UC-10 Decide duplicate case	—	✓	—	«include» append review event; «include» write audit entry
UC-11 Re-scan existing registry	—	✓	—	«include» score all unflagged pairs
UC-12 Decide award (approve or reject)	—	✓	—	«include» check verification; «include» check open duplicates
UC-13 Record payment as issued	✓	✓	—	«include» check approved award; «include» check duplicates; «include» check amount
UC-14 Clear payment	✓	✓	—	«include» require evidence reference
UC-15 Acknowledge receipt	✓	✓	—	«extend» flag amount discrepancy
UC-16 Review reconciliation exceptions	✓	✓	—	—
UC-17 Issue award certificate	—	✓	—	«include» require approved award
UC-18 Revoke award certificate	—	✓	—	«include» require reason
UC-19 Read or print certificate	✓	✓	—	—
UC-20 View dashboard	—	✓	—	—
UC-21 Export PDF report	—	✓	—	—
UC-22 View audit log	—	✓	—	—
UC-23 Manage thresholds	—	✓	—	«include» write audit entry
UC-24 Manage user accounts	—	✓	—	—
UC-25 Verify student QR	—	—	✓	—
UC-26 Verify certificate QR	—	—	✓	—

Two generalisations should be drawn explicitly: *Administrator* and *Field Officer* are both specialisations of an abstract *Authenticated staff user*, because both sign in and both read their own profile; and the two public use cases

(UC-25, UC-26) are the only use cases reachable without authentication. Because a use case diagram cannot express the restriction that matters most — that an administrator cannot register a student and a field officer cannot decide a duplicate case — Table 4.6 is the authoritative statement of those rules, and the figure should carry a note directing the reader to it.

4.3.3 Activity diagram: registration and duplicate screening

This is the system's most consequential process, because it is the point at which a machine score influences an administrative workflow. The model must show that the student record is created **whatever the screening result**, and that the only effect of a positive result is the creation of review cases.

[Insert Figure 4.3: Activity diagram for beneficiary registration and duplicate screening here]

Figure 4.3: Activity diagram — registration and duplicate screening. Swimlanes: *Field Officer*, *API (students controller)*, *Duplicate predictor*, *PostgreSQL*. The activity flow must contain these steps in this order:

1. Field Officer submits the registration form (first name, last name, date of birth, school, national identifier, optional birth-certificate number, optional UPI, phone).
2. API validates the payload (§4.2.4); on failure it returns 400 with field-level errors and the flow ends.
3. API resolves the review threshold and the aggregation policy **before** opening a transaction, so a misconfiguration fails without holding a database transaction open.
4. API generates a 64-hex QR token and builds the verification URL from the validated `PUBLIC_APP_URL`; a bad origin aborts the request here.
5. API begins a transaction and takes the advisory lock `sbms.student.registration`.
6. API selects every existing student `FOR UPDATE`.
7. API computes six features for each candidate and calls the predictor in batches of 200.
8. Decision: predictor unavailable, timed out or returned an invalid payload → roll back, return 503, and no student is created.
9. Decision per candidate: score ≥ review threshold, or an exact UPI / birth-certificate / national identifier agreement → candidate qualifies; otherwise it does not.
10. Aggregation decision point (policy `top-1` by default): among qualifying candidates, rank exact identifier agreement above score and select at most one.
11. API inserts the student row, then inserts one `duplicate_flags` row for each selected candidate, with score, model version, threshold and feature vector.
12. API writes the audit entry, including candidates considered, qualifying and suppressed, and commits.
13. API returns 201 with the student (without the QR token), the QR image and the screening summary; the Field Officer sees either "no match", "possible duplicate match found — an administrator must review this case", or the exact-identifier sentence.

The diagram must show the aggregation decision as a **policy node**, not a model node, and a note must record that no branch of the flow deletes, rejects or edits a beneficiary.

4.3.4 Activity diagram: duplicate review, award approval and payment gate

The second activity model explains how a review case propagates into money. The property that matters is that a student with an **open, further-verification or confirmed** case cannot be approved or paid, and that only an administrator can release the hold.

[Insert Figure 4.4: Activity diagram for duplicate review, award approval and the payment gate here]

Figure 4.4: Activity diagram — review, approval and payment gate. Swimlanes: *Administrator*, *Field Officer*, *API*, *PostgreSQL*. Required structure:

1. Administrator opens the duplicate queue; the API returns each case with masked identifiers, model score, model version and the threshold in force.

2. Decision: are the full identifiers needed to compare source documents? - **Yes** → reveal request → API writes a `REVEAL_DUPLICATE_IDENTIFIERS` audit row and returns the five identifier fields → continue. - **No** → continue with masked values.
3. Administrator records a decision with a reason of at least five characters. The API updates the case only from `pending` or `needs_verification`, appends a `duplicate_review_events` row and writes an audit entry. A terminal case cannot be re-decided (404).
4. Decision per outcome: - `not_duplicate` → the hold is cleared for that pair; - `needs_verification` → the hold persists and a later decision may still be recorded; - `confirmed_duplicate` → the hold persists permanently and cannot be reversed through the API.
5. Field Officer submits an award (`requested_amount`) and records a beneficiary verification outcome. A `verified` outcome without an evidence reference is refused (400).
6. Administrator approves or rejects. Approval is refused with 409 unless (a) the beneficiary is verified and (b) no case against that student is `pending`, `needs_verification` or `confirmed_duplicate`, where the student may be either side of the pair.
7. Field Officer or Administrator records a payment against the approved award. The API re-checks award status, verification status, the duplicate hold and the remaining approved balance, and refuses a payment exceeding the balance (409).
8. Payment transition: `issued` → `cleared` (evidence required), or `issued` → `failed` / `cancelled` (reason of at least five characters required).
9. Receipt acknowledgement: the officer records the amount actually received; a shortfall or excess is stored as `discrepancy` and appears in the exceptions list rather than being corrected silently.
10. Every step from 5 to 9 writes both a domain event row and an `audit_logs` row.

Step 7 must be drawn as a **second, independent gate** rather than a consequence of step 6, because the implementation re-validates at payment time and a case raised after approval must still block payment.

4.3.5 Class diagram

The class model describes the application-level domain and its behaviour, not the database tables (those are modelled in §4.3.10).

[Insert Figure 4.5: Class diagram of the SBMS domain here]

Figure 4.5: Class diagram. Classes, principal attributes and operations, with multiplicities:

Class	Principal attributes	Principal operations	Relationships
`User`	id, full_name, email, password_hash, role, is_active, created_at	`findByEmail`, `createUser`, `authenticateToken`, `requireRole`	1 User creates 0..* Student; resolves 0..* DuplicateFlag; decides 0..* BursaryAward; issues 0..* Certificate
`Student`	id, full_name, date_of_birth, school_code, guardian_id, upi, birth_certificate_number, national_id_number, primary_phone, qr_token, is_synthetic, eligibility_score, eligibility_category, created_at	`create`, `getById`, `getAll`, `hasQrToken`, `withoutQrToken`	participates in 0..* DuplicateFlag (as subject or as match); holds 0..* BursaryAward
`DuplicateFlag`	id, student_id, matched_student_id, probability, status, model_version, review_threshold, comparison_features, resolved_by, resolved_at, updated_at	`list`, `evaluate`, `scanExisting`, `update`, `revealIdentifiers`	belongs to 2 Students; has 1..* DuplicateReviewEvent
`DuplicateReviewEvent`	id, duplicate_flag_id, reviewer_id, decision, reason, notes, created_at	append-only	belongs to 1 DuplicateFlag and 1 User
`BursaryAward`	id, student_id, school_code, requested_amount, approved_amount, status, verification_status, verification_evidence_ref, verified_by, verified_at, decision_reason, decided_by, decided_at	`list`, `create`, `recordVerification`, `decide`	1 Student holds 0..* BursaryAward; 1 BursaryAward has 0..* Disbursement and 0..1 Certificate

Class	Principal attributes	Principal operations	Relationships
`Disbursement` (payment)	id, student_id, award_id, amount, payment_method, payment_reference, payment_status, clearance_evidence_ref, receipt_status, acknowledged_amount, acknowledgement_method	`list`, `listExceptions`, `create`, `updatePaymentStatus`, `acknowledgeReceipt`, `linkLegacyPayment`	belongs to 1 Student and 0..1 BursaryAward; has 0..* ReconciliationEvent
`BursaryCertificate`	id, award_id (unique), certificate_reference, verification_token, award_period, student_name, student_upi, school_code, approved_amount, award_approved_at, status, issued_by, issued_at, revoked_by, revoked_at, revocation_reason	`insert`, `findByAwardId`, `findByVerificationToken`, `revoke`, `issue`, `downloadPdf`, `verifyPublic`	belongs to exactly 1 BursaryAward
`AuditEntry`	id, user_id, action, table_name, record_id, details, created_at	`logAudit`	belongs to 0..1 User
`DuplicatePredictor` (service)	model file, SHA-256 version, feature order	`scoreStudentPairs`, `predict_pairs`, `model_version`	called by `Student` registration and by `DuplicateFlag` rescan
`AggregationPolicy` (service)	policy name (`top-1`, `top-1-corroborated`, `all`), threshold	`selectFlaggedCandidates`, `summarizeAggregation`	used by `Student.create`

Three design points must be annotated on the figure. First, `DuplicateFlag.probability` is a legacy column name: for model-created rows it holds an **uncalibrated positive-class score**, and the schema comment says so. Second, no class exposes an operation that rejects or deletes a student; the only student-mutating operations are creation and eligibility update. Third, `BursaryCertificate` duplicates award facts on purpose, so a later award edit cannot rewrite a printed document.

4.3.6 System sequence diagram: registration

A system sequence diagram (SSD) treats the system as a black box and shows only the actor-to-system messages for one scenario, establishing the system operations that the detailed sequence diagrams then realise.

[Insert Figure 4.6: System sequence diagram for beneficiary registration here]

Figure 4.6: System sequence diagram — register a beneficiary. Actors: *Field Officer* (left), *:SBMS* (right, a single lifeline). Messages, in order:

1. `openRegistrationPage()` → the system returns the form and populates the school list from `GET /api/students/schools`.
2. `submitRegistration(firstName, lastName, dateOfBirth, school, nationalId, [birthCertificateNumber], [upi], phone)`.
3. The system returns `201 {student, qr_code, duplicate_flags, duplicate_screening}`, **or** `400` with field errors, **or** `503` when screening is unavailable.
4. `viewScreeningResult()` → the officer reads one of three states: no candidate; a possible duplicate match requiring administrator review; or an exact identifier match named in plain language.
5. `printQrCode()` → the officer prints or displays the QR image returned in step 3.

The SSD must show an **alternate frame** for the 503 outcome: registration is refused, no student record exists afterwards, and the officer is told to retry. A note must state that the model score and threshold are deliberately absent from the officer's view.

4.3.7 Sequence diagram: duplicate review decision

[Insert Figure 4.7: Sequence diagram for an administrator's duplicate review decision here]

Figure 4.7: Sequence diagram — decide a duplicate case. Lifelines: *Administrator*, *DuplicateReview (React)*, *duplicateFlags.controller*, *PostgreSQL*. Messages:

1. `GET /api/duplicate-flags` → the controller authenticates, checks `admin`, queries the joined case list, masks `guardian_id`, `upi`, `birth_certificate_number` and `national_id_number` to their last three characters, and returns the policy block (review threshold, high-priority threshold, aggregation policy, `thresholds_validated: false`).
2. `PATCH /api/duplicate-flags/:id {decision, reason, notes}` → the controller validates the decision enumeration and a reason of at least five characters.
3. `BEGIN`; `UPDATE duplicate_flags SET status, resolved_by, resolved_at WHERE id AND status IN ('pending','needs_verification') RETURNING *`.
4. **alt** no row updated → `ROLLBACK`, `404 {message:'Duplicate flag not found'}`.
5. `INSERT INTO duplicate_review_events (duplicate_flag_id, reviewer_id, decision, reason, notes)`.
6. `INSERT INTO audit_logs (… action = 'DUPLICATE_' || decision …)` carrying the model version, the score and the review threshold that created the case.
7. `COMMIT`; respond `200 {message, flag}`.
8. Optional earlier message: `GET /api/duplicate-flags/:id/identifiers` → the controller returns the five identifier fields for both students and writes a `REVEAL_DUPLICATE_IDENTIFIERS` audit row.

The reveal must be drawn as a **separate interaction with its own audit write**, because the design decision is that disclosure is an event, not a field permission.

4.3.8 Sequence diagram: award approval, payment and certificate issuance

[Insert Figure 4.8: Sequence diagram for award approval, payment reconciliation and certificate issuance here]

Figure 4.8: Sequence diagram — approval to certificate. Lifelines: *Field Officer*, *Administrator*, *bursaryAwards.controller*, *disbursements.controller*, *certificates.controller*, *PostgreSQL*. Messages:

1. `POST /api/awards {student_id, requested_amount}` (field officer) → the student must exist and must have a school; the award row is created with `status='submitted'` and `verification_status='not_checked'`.
2. `POST /api/awards/:id/verification {outcome, notes, evidence_ref}` (field officer) → a `verified` outcome without an evidence reference is refused (`400`); an event row is appended and the award's verification fields updated.
3. `PATCH /api/awards/:id/decision {decision:'approved', approved_amount, reason}` (administrator) → advisory lock; `SELECT … FOR UPDATE`; status must be `submitted`; verification must be `verified`; no `pending`, `needs_verification` or `confirmed_duplicate` case may exist on either side of the student's pairs; then `status='approved'` and an `AWARD_APPROVED` audit row.
4. `POST /api/disbursements {award_id, amount, payment_method, payment_reference, disbursement_date}` (either role) → advisory lock; the award must be `approved` and `verified`; the duplicate check is repeated; `SUM(existing amounts) + amount` must not exceed `approved_amount`; insert with `payment_status='issued'` and `receipt_status='awaiting_acknowledgement'`; append a `payment_issued` event; audit `PAYMENT_ISSUED`.
5. `PATCH /api/disbursements/:id/payment-status {payment_status:'cleared', evidence_ref}` → transition from `issued` only; evidence required; `payment_cleared` event; audit `PAYMENT_CLEARED`.
6. `PATCH /api/disbursements/:id/acknowledgement {acknowledged_amount, evidence_ref}` → compares the payment and acknowledged amounts in integer cents; `acknowledged` when equal, `discrepancy` otherwise; `receipt_acknowledged` event; audit `RECEIPT_ACKNOWLEDGED_OFFICER_ASSISTED`.
7. `POST /api/awards/:id/certificate {award_period?}` (administrator) → the award must be `approved` and must not already have a certificate; a snapshot row is inserted with a fresh 64-hex token and an `SBMS-CERT-<year>-<8 hex>` reference; `AWARD_CERTIFICATE_ISSUED` audit row.
8. `GET /api/awards/:id/certificate.pdf` (either role) → PDF rendered per request carrying the same QR code; `Cache-Control: no-store`.

9. `POST /api/awards/:id/certificate/revoke {reason}` (administrator) → status `active` → `revoked` with actor, timestamp and reason; `AWARD_CERTIFICATE_REVOKED` audit row; the award is unchanged.

Two distinctions must be visually explicit: **approval is not payment** (steps 3 and 4 are separate lifelines with separate audit actions), and **certificate verification is not student verification** (step 7 uses the certificate token namespace, not the student QR token).

4.3.9 Sequence diagram: public QR verification

[Insert Figure 4.9: Sequence diagram for public student-record and certificate verification here]

Figure 4.9: Sequence diagram — public verification. Lifelines: *Public verifier (phone browser)*, *Verification page*, *verification.controller*, *certificates.controller*, *PostgreSQL*. Messages:

1. The verifier scans the printed code; the phone opens `https://<PUBLIC_APP_URL>/verify#token=<64 hex>` or `/verify-certificate#token=<64 hex>`.
2. The page reads the fragment, removes it with `history.replaceState`, and (re)runs verification on `hashchange` so a second scan in the same tab cannot display the previous result.
3. `POST /api/verify {token}` → rate limiter runs **before** the JSON parser; the controller requires exactly one key named `token` matching `^[a-f0-9]{64}$`; `SELECT EXISTS(… WHERE qr_token = $1)`.
4. Response `{found:true}` or `{found:false}`, and nothing else; malformed and unknown tokens are indistinguishable; a lookup failure returns `503 {error:'temporarily_unavailable'}`; the eleventh request in the window returns `429`.
5. `POST /api/certificates/verify {token}` → same shape discipline with its own 20-per-15-minute budget; the controller loads the certificate together with the live award and returns one of four states: `verified`, `revoked`, `not_current`, `not_recognised`.
6. The page renders a status-only view: no student name, UPI, school, date of birth, identifier, phone or amount appears in any response or on any page.

The figure must show the rate limiter as the **first** participant after the page, because the implementation registers the limiter in the same route chain ahead of the body parser, and must show the two endpoints as independent chains with independent budgets.

4.3.10 Entity-relationship diagram

[Insert Figure 4.10: Entity-relationship diagram of the SBMS database here]

Figure 4.10: Entity-relationship diagram. Chen notation with cardinalities; every attribute shown must exist in `backend/src/db/schema.sql` or in one of the migrations listed in §4.6.1. Entities and relationships:

Entity	Relationship	Entity	Cardinality	Participation
`users`	creates	`students`	1 : N	Student creation is not stored as a foreign key on `students`; the creating user is recorded in `audit_logs`
`students`	is subject of	`duplicate_flags`	1 : N	Total on the flag side (`student_id NOT NULL`)
`students`	is matched by	`duplicate_flags`	1 : N	Total on the flag side (`matched_student_id NOT NULL`)
`users`	resolves	`duplicate_flags`	1 : N (nullable)	Partial: `resolved_by` is null until a decision
`duplicate_flags`	has	`duplicate_review_events`	1 : N	Total on the event side
`users`	reviews	`duplicate_review_events`	1 : N	Total
`students`	holds	`bursary_awards`	1 : N	Total; `ON DELETE RESTRICT`
`users`	submits / verifies / decides	`bursary_awards`	1 : N (nullable)	Partial per role column

Entity	Relationship	Entity	Cardinality	Participation
`bursary_awards`	has	`bursary_award_verification_events`	1 : N	Total
`students`	receives	`disbursements`	1 : N	Total
`bursary_awards`	is paid by	`disbursements`	1 : N (nullable)	Partial: legacy payments have no `award_id`
`disbursements`	has	`disbursement_reconciliation_events`	1 : N	Total
`bursary_awards`	is certified by	`bursary_certificates`	1 : 0..1	`award_id UNIQUE`
`users`	issues / revokes	`bursary_certificates`	1 : N (nullable for revoke)	Partial
`users`	performs	`audit_logs`	1 : N (nullable)	Partial: system actions may have no user
`users`	generates	`reports`	1 : N (nullable)	Partial

The figure must carry three annotations. The `beneficiaries` table stands **outside** the modelled relationships: it has a serial integer key, no foreign keys, and is retained only because the legacy endpoint and the readiness check require it. The `students.qr_token` attribute has a **partial unique index**, not a table constraint. `duplicate_flags` carries a unique index over the **unordered pair** `(LEAST(student_id, matched_student_id), GREATEST(student_id, matched_student_id))`, which is what guarantees that the same two students cannot generate two cases.

4.3.11 Machine-learning duplicate-detection pipeline

Because the research contribution is the screening pipeline rather than the CRUD application, the pipeline is modelled separately from the application workflow.

[Insert Figure 4.11: Machine-learning duplicate-detection data pipeline here]

Figure 4.11: ML duplicate-detection pipeline. The figure must be drawn in two lanes, *offline (training and evaluation)* and *online (serving)*, joined only at the shared feature module, because that shared module is the property that makes the served score the same quantity the metrics describe.

Offline lane: generated record pairs → corruption scenarios (`repeat_application`, `siblings_shared_household`, `twins_or_close_age`, `common_name_same_school`, `identifier_collision`, `unrelated_candidate`) → `feature_engineering.build_pair_features` (six features) → grouped 80/20 split by identity → XGBoost training with the selected hyperparameters (`n_estimators 250`, `max_depth 4`, `learning_rate 0.05`, `subsample 0.9`, `colsample_bytree 0.9`, `reg_lambda 1.0`) → independently generated holdout of 12 000 pairs → evaluation harness (`evaluate_operating_point.py`, `evaluate_false_positive_rate.py`) → metrics report and operating-point evidence → artefact `model/xgb_duplicate_model.joblib` with SHA-256 identity.

Online lane: `POST /api/students` → candidate set (all existing students) → `scorePairs` batching at 200 pairs → predictor process (`predict.py --server`, one process, line-delimited JSON) → `build_pair_features` per pair → `predict_proba[:,1]` → score per pair streamed back → aggregation policy (`top-1`) → `duplicate_flags` row with score, model version, threshold and feature vector → administrator review queue → human decision.

The figure must also show, as a separate off-page artefact, `model/research/rf_attachment_style_candidate.joblib` and `experiments/sbms_dataset/artifacts/sbms_dataset_xgb_research_only.joblib`, marked **not loaded by the application**, because conflating them with the production artefact would misstate the system's evidence base.

4.4 System Design

4.4.1 Overall architectural design

SBMS follows the three-tier client–server architecture named in the approved proposal (§3.4.5), with one deliberate refinement: the machine-learning component is **not** a separate network service. It runs as a long-lived child process of the API, driven over standard input and output.

[Insert Figure 4.12: Three-tier architecture and component deployment of SBMS here]

Figure 4.12: System architecture. Tiers and components:

Tier	Component	Technology	Responsibility
Presentation	Single-page application	React 18.3, React Router 7, Tailwind CSS 4, `chart.js` + `react-chartjs-2` for dashboard charts, `lucide-react` icons, `axios` for HTTP	Eighteen routes: two public verification pages, three sign-in routes and role-protected operational screens; the session token is held in `localStorage` and attached by an interceptor
Application	REST API	Node.js 22 (`node:22-bookworm-slim`), Express 4, `zod`, `bcrypt`, `jsonwebtoken`, `pg`, `qrcode`, `pdfkit`	Authentication, authorisation, validation, duplicate-case lifecycle, award and payment rules, certificate issue and revocation, reporting, audit
Application (sub-process)	Duplicate predictor	Python 3 in a `venv`, `pandas`, `scikit-learn`, `xgboost`, `jellyfish`	Loads one artefact, builds six features per pair, returns `predict_proba` scores and the artefact hash
Data	Relational database	PostgreSQL 16 (managed Flexible Server in Azure; 17.11 on the development machine)	Durable state, referential integrity, `CHECK` constraints, unique indexes

Three architectural decisions require justification because they are not the obvious choices.

The ML component is a child process, not an HTTP service. The predictor is started once with `predict.py --server`, kept alive and addressed with one JSON line per request (`services/duplicateCheck.js:260-290`). This avoids a second network hop, a second authentication problem and a second deployment unit, and it is why the whole system ships as **one container image**. The cost is that the predictor's lifetime is coupled to the API process: if it exits, every pending request is rejected and the next request respawns it (NFR-20). The same decision means the API cannot be scaled horizontally without giving each replica its own predictor and its own copy of the in-memory rate-limit counters (NFR-07).

Two separately deployed web units. The frontend is a static bundle served by App Service and the API is a container, joined by `VITE_API_ORIGIN` at build time and by `CORS_ORIGINS` at run time. This was forced by the hosting constraint that Static Web Apps were unavailable in the only permitted Azure regions (`docs/azure-deployment-log-2026-10-31.md` §2), and it has a real consequence recorded in the same log: a relative `fetch("/api/verify")` from the static host never reached the API, which was a genuine production defect until the call was routed through the axios client.

Migrations are never applied automatically. The API validates the schema it needs at start-up and exposes readiness, but applying SQL is an explicit, backed-up operator action (`backend/scripts/migrate.js`; `README.md` "Database operations"). This is a safety decision for a system holding beneficiary records, and it is why `/api/ready` fails closed when a migration is missing.

4.4.2 Backend module design

The backend is organised by responsibility rather than by feature, and the boundaries are visible in what each module is allowed to do.

Table 4.4: Backend modules and responsibilities

Layer	Modules	Responsibility	Notes
Entry	`src/index.js`	Security headers, CORS, proxy trust, body parsing, public route chains, route mounting, health and readiness, fail-closed start-up	Start-up validates JWT secret, `PUBLIC_APP_URL` and the authentication schema before listening
Configuration	`config/db.js`, `jwtSecret.js`, `publicAppUrl.js`, `cors.js`, `settings.js`, `schools.js`, `reconciliation.js`	Pool, secret validation, public-origin validation, allowlist parsing, live thresholds, approved-school list, acknowledgement deadline	Thresholds are held in memory and are process-local
Middleware	`auth.js`, `requireRole.js`, `loginRateLimit.js`, `verificationRateLimit.js`, `certificateRateLimit.js`, `rateLimitWindow.js`, `errorHandler.js`, `asyncHandler.js`	Authentication, authorisation, rate limiting, sanitised errors, async error propagation	Each limiter owns a separate counter map
Routing	Twelve `routes/*.routes.js` files	Endpoint declaration and role gates only	One `requireRole(...)` per route
Controllers	Fourteen `controllers/*.controller.js` files	Validation, transaction boundaries, domain sequencing, response shapes	Rules that must be reused live in services
Services	`duplicateCheck.js`, `mlDuplicateReview.js`, `duplicateAggregation.js`, `bursaryRules.js`, `eligibility.js`, `certificateDocument.js`, `auditLog.js`, `studentRecordFormat.js`, `databaseReadiness.js`, `databaseStartup.js`	Predictor bridge, batching and priority, aggregation policy, payment state machine and exception rules, eligibility scoring, PDF rendering, audit writes, normalisation, readiness and start-up assertions	`duplicateCheck.js` also retains a deprecated rule engine that no request path calls
Models	`auth.model.js`, `student.model.js`, `disbursement.model.js`, `bursaryCertificate.model.js`, `beneficiary.model.js`, `audit.model.js`	Parameterised SQL only	The student model strips `qr_token` before returning a record
Scripts	`migrate.js`, `bootstrap-production-schema.js`, `provision-test-database.js`, `seed-*.js`, `reset-*.js`, `kenyanSyntheticDataset.js`, `cleanup-legacy-school-data.js`, `inspect-render-db.js`	Migration runner, guarded empty-schema bootstrap, disposable test database, synthetic demonstration data	The empty-schema bootstrap refuses a non-identical target and refuses existing relations

Two design weaknesses are recorded here rather than hidden. First, `duplicateCheck.js` still exports a weighted rule engine (`buildDuplicateFeatures`, `calculateDuplicateScore`, `checkDuplicates`) that has **no production caller**: it is retained for historical comparison and is exercised only by tests, so it is dead weight a maintainer could mistake for the live scoring path. Second, the `/api/beneficiaries` router and `beneficiary.model.js` persist a legacy table whose semantics are not reconciled with the student model, so two partially overlapping notions of "beneficiary" coexist in one schema.

4.4.3 Authentication, authorisation and access-control design

Authentication is a two-step design in which the token proves *who signed in* and the database proves *what that account may do now*. On sign-in the API looks the account up by e-mail, rejects it if `is_active` is false or the bcrypt comparison fails, returns 403 if the account's role does not match the endpoint the user chose, and otherwise signs `{id, role}` with an eight-hour expiry. On every later request the middleware verifies the signature and expiry, then re-reads `id, role` from `users` **and requires the row to still be active** (`middleware/auth.js:17-25`). A signed token therefore cannot outlive a revoked role or a deactivated account, which is the failure mode an eight-hour JWT would otherwise introduce.

Table 4.5: Role model

Role	Value	Population route	Purpose
Administrator	`admin`	Created by an existing administrator through `POST /api/users`	Owns duplicate decisions, award decisions, thresholds, reports, audit review, certificate issue and revocation
Field Officer	`field_officer`	Created by an administrator; the public registration route always answers 410	Owns registration, eligibility assessment, award submission, beneficiary verification, payment recording, certificate reading and printing
Public verifier	*(unauthenticated)*	Not an account	May call only `POST /api/verify` and `POST /api/certificates/verify`

Table 4.6: Access matrix for authenticated endpoints

Endpoint	Method	Admin	Field Officer	Notes
`/api/auth/me`	GET	✓	✓	Authenticated profile
`/api/auth/register`	POST	410	410	Public registration disabled
`/api/users`	GET, POST	✓	403	Administrative provisioning
`/api/students` and `/api/students/:id`, `/api/students/schools`	GET, POST	403	✓	Deliberate: administrators receive beneficiary data from the endpoints built for their screens
`/api/duplicate-flags`	GET	✓	403	Masked identifiers
`/api/duplicate-flags/:id/identifiers`	GET	✓	403	Audited reveal
`/api/duplicate-flags/:id`	PATCH	✓	403	Decision
`/api/duplicate-flags/evaluate`	POST	✓	403	Paired scoring without persistence
`/api/duplicate-flags/scan`	POST	✓	403	Registry rescan
`/api/awards`	GET	✓	✓	
`/api/awards`	POST	403	✓	Submission is a field activity
`/api/awards/:id/verification`	POST	403	✓	
`/api/awards/:id/decision`	PATCH	✓	403	Approval is administrative
`/api/awards/:id/certificate`	POST	✓	403	Issue
`/api/awards/:id/certificate`, `…/certificate.pdf`	GET	✓	✓	Read and print
`/api/awards/:id/certificate/revoke`	POST	✓	403	
`/api/disbursements`, `/exceptions`, `/:id/payment-status`, `/:id/acknowledgement`	GET, POST, PATCH	✓	✓	Both roles may record and clear a payment
`/api/disbursements/:id/award`	PATCH	✓	403	Linking a legacy payment is administrative
`/api/eligibility`	POST	403	✓	
`/api/reports/export`	GET	✓	403	
`/api/audit-logs`	GET	✓	403	
`/api/settings/threshold`	GET, PUT	✓	403	
`/api/admin/dashboard`	GET	✓	403	

The `/api/students` row is the one that most often looks like a defect, and it is worth stating why it is not. Administrators own duplicate review, awards, reports and the dashboard, but each of those screens obtains the beneficiary data it displays from an endpoint built for that screen: the dashboard receives aggregates, the review queue receives the two records belonging to a case with identifiers masked, the awards and payments screens receive joined student identity, and the report receives an aggregate. No administrative screen calls `/api/students`, and the one shared screen, `Disbursements.jsx`, requests the student list only when the signed-in user is a field officer (`frontend/src/pages/Disbursements.jsx:52-53`). The restriction is therefore the designed policy, it is pinned by an explicit regression test (`tests/roleAuthorization.test.js`, "student routes stay field-officer-only while admin

data paths keep working"), and the observation that produced the test is recorded in `SBMS_O10_O11_IMPLEMENTATION_REPORT.md` §2.1 and §5.

4.4.4 Machine-learning integration design

The integration design has four parts: the boundary, the score, the decision rule around the score, and the record of what happened.

The boundary. The model never sees a database row. The controller selects candidates, maps each into two plain objects with exactly the fields the feature builder reads, and hands them to `scorePairs` (`controllers/students.controller.js:69-73`; `services/mlDuplicateReview.js:84-117`). Batches are capped at 200 pairs, results are checked for count, range and stability of `model_version`, and any violation raises rather than silently continuing. The predictor returns `{score, features}` per pair, so a case can be re-derived later without the predictor.

The score. The score is the positive-class probability of the classifier, rounded to four decimal places, and it is **not calibrated**: the code publishes `score_is_calibrated_probability: false` and the schema comment on `duplicate_flags.probability` records that model-created rows hold an uncalibrated score. Three settings surround it:

Setting	Environment variable	Default	Meaning
Review cut-off	`ML_REVIEW_SCORE_THRESHOLD`	`0.50`	A pair at or above this score becomes a review candidate
High-priority cut-off	`DUPLICATE_HIGH_PRIORITY_THRESHOLD`	`0.70`	Orders a case as *high* priority; presentation only
Legacy review threshold	`DUPLICATE_REVIEW_THRESHOLD`	`0.30`	Published for cases created before provenance columns existed
Aggregation policy	`DUPLICATE_AGGREGATION_POLICY`	`top-1`	How many qualifying candidates become cases

The decision rule around the score. This is the part of the design that changed materially during the project, and the change is itself a design finding. Originally every qualifying pair created a flag; because the candidate set is the whole registry, the probability that at least one nuisance flag appears grows as `1 − (1 − p)^N`, which at `p = 0.35%` and `N = 1 000` is about 97% (`services/duplicateAggregation.js:8-14`). The design now separates *scoring* from *case selection*:

- the model decides a per-pair score;
- `qualifiesForReview` decides whether a candidate qualifies, either by score or by an exact identifier agreement the model does not see;
- `selectFlaggedCandidates` ranks qualifiers with exact identifier agreement above score and, under the default policy, returns at most one;
- `summarizeAggregation` records how many candidates were considered, how many qualified and how many were suppressed, so suppression is auditable rather than invisible.

Three policies exist (`top-1`, `top-1-corroborated`, `all`). `all` reproduces the earlier behaviour for comparison; `top-1-corroborated` is marked experimental in the code and the project's own measurement recommends against it, because it removes 34–54% of nuisance flags at the cost of 39–60% of true duplicates (`docs/ml-decision-sheet-2026-10-30.md` §5).

The record. Each case stores the score, the artefact hash as `model_version`, the review threshold under which it was created and the six-feature vector, and the registration audit row stores the aggregation policy and the three candidate counters. A reviewer therefore sees not only "this was flagged" but "flagged by this artefact, at this threshold, under this policy, from this vector, with this many other candidates considered".

What the design deliberately does not do. It does not block, reject, merge, delete or edit a beneficiary; it does not restrict the comparison set (the whole registry is compared); it does not use the national identifier as a feature; and it does not retrain itself. Each is a scope decision recorded in Chapter Five §5.6 and returned to in Chapter Six.

4.4.5 Workflow and state design

Four state machines carry the business rules. They are specified here because Chapter Five asserts transitions, not prose.

Table 4.7: Duplicate-case states

State	Entered by	Permitted next states	Effect on payment
`pending`	Registration or registry rescan creates the case	`confirmed_duplicate`, `not_duplicate`, `needs_verification`	Blocks approval and payment
`needs_verification`	Administrator decision	`confirmed_duplicate`, `not_duplicate`, `needs_verification` (again)	Blocks approval and payment
`confirmed_duplicate`	Administrator decision	terminal (a repeat decision returns 404)	Blocks approval and payment permanently
`not_duplicate`	Administrator decision	terminal	Holds cleared for that pair

Every decision writes a `duplicate_review_events` row whose `decision` is constrained to the three decision values and whose `reason` must be non-blank (`schema.sql`, `duplicate_review_events`), so a case returned to review is still recorded as an event rather than losing its history. A unique index over `LEAST(student_id, matched_student_id)` and `GREATEST(...)` (`duplicate_flags_unordered_pair_unique`) means a pair can hold only one case, whichever direction it was first detected in.

Table 4.8: Bursary-award states

State	Entered by	Conditions	Permitted next states
`submitted`	Field officer submission	Student exists and has a school; `requested_amount > 0`	`approved`, `rejected`
`approved`	Administrator decision	Verification `verified`; no open or confirmed duplicate for the student; `approved_amount` supplied; reason ≥ 5 characters	terminal
`rejected`	Administrator decision	Reason ≥ 5 characters; no amount recorded	terminal

A database constraint enforces the field combination for each state (`bursary_awards_decision_fields_check`), and a second refuses an `approved` row whose verification status is not `verified` (`bursary_awards_approval_verification_check`).

Table 4.9: Payment and receipt states

Field	States	Rules
`payment_status`	`issued` → `cleared` / `failed` / `cancelled`	Only `issued` is reachable on creation; clearance requires `cleared_by`, `cleared_at` and a non-empty `clearance_evidence_ref` (`disbursements_clearance_fields_check`)
`receipt_status`	`NULL` → `awaiting_acknowledgement` → `acknowledged` / `discrepancy`	Acknowledgement records the amount received, the method (`officer_assisted`) and an evidence reference; an unequal amount becomes `discrepancy`, never a correction of the recorded payment

Table 4.10: Certificate states

State	Entered by	Public verification answer
(no certificate)	—	`not_recognised`
`active`	Administrator issue	`verified` when the award is still `approved` at the same amount; `not_current` otherwise
`revoked`	Administrator revoke with a reason	`revoked`

The certificate transition is one-way: a revoked certificate cannot be reissued for the same award, because `award_id` is unique and the issue path refuses an award that already has a certificate row.

4.4.6 Security and privacy design

Security was designed around four threats implied by the project's own problem statement: disclosure of beneficiary data, unauthorised payment, unauthorised disclosure through a printed code, and silent tampering with a decision.

Threat	Control	Where
Credential theft by offline attack	bcrypt cost 12; the API refuses to start with a missing, short or placeholder JWT secret	`controllers/auth.controller.js`; `config/jwtSecret.js`
Credential stuffing	10 sign-in attempts per IP per 15 minutes	`middleware/loginRateLimit.js`
Privilege escalation by forged claim	Role re-read from the database on every request	`middleware/auth.js`
Unauthorised payment	Two independent gates (approval and payment) plus an approved-balance check; both gates query both sides of a duplicate pair	`controllers/bursaryAwards.controller.js:173-189`; `controllers/disbursements.controller.js`
Data disclosure in the review queue	Identifier masking; reveal is a separate audited action; the student model strips `qr_token` from every response	`controllers/duplicateFlags.controller.js`; `models/student.model.js`
Disclosure through a printed code	Token in the URL fragment; public endpoint answers one boolean; no beneficiary field in any public response	`config/publicAppUrl.js`; `controllers/verification.controller.js`
Enumeration through a public endpoint	Fixed token shape; identical answer for malformed and unknown tokens; per-IP budget; separate budgets per endpoint	`middleware/rateLimitWindow.js`; `controllers/certificates.controller.js:248-270`
Tampering with an issued document	Certificate snapshot compared with the live award; revocation recorded with actor, time and reason	`models/bursaryCertificate.model.js`; migration `20261031_bursary_certificates.sql`
Information leakage in errors	5xx responses carry a fixed message; the handler logs an allowlisted diagnostic only	`middleware/errorHandler.js`
Cross-site data access	CORS allowlist plus a pre-routing origin check; security headers on every response	`index.js:33-47`; `config/cors.js`
Bulk or abusive requests	100 kB JSON limit; oversize rejected before the handler	`index.js:64`

Two controls a reviewer might expect are **absent**, and are limitations rather than implied features: there is no multi-factor authentication, and the application applies no encryption at rest of its own (it relies on the managed database service). Sessions are also not individually revocable, because a JWT remains valid until expiry unless the account is deactivated.

4.4.7 Validation and error-handling design

Four conventions apply throughout.

1. **Validation precedes side effects.** Schemas are evaluated before a transaction is opened, and the threshold and aggregation policy are resolved before the connection is taken, so a misconfiguration never leaves a transaction open (`controllers/students.controller.js:54-57`).
2. **Domain refusals are 409, not 400.** "Verify the beneficiary before approving this award", "Resolve duplicate review cases before approving this award" and "Only verified, approved awards can be disbursed" describe conflicts with current state and are expected to be actionable by the client (`controllers/bursaryAwards.controller.js:169-188`).
3. **Public endpoints never distinguish "malformed" from "unknown".** Both produce the same answer, so the endpoint cannot be used to test whether a token was ever issued.
4. **Server errors are opaque.** A sanitised handler returns `{message:'Internal server error'}` for any 5xx and logs only an allowlisted diagnostic, so a stack trace, SQL fragment or credential cannot reach a client or a log

aggregator (`middleware/errorHandler.js:59-78`; `tests/securityMiddleware.test.js`).

4.5 System Design Diagrams (interface design)

The interface design is presented as screen specifications rather than as decorative wireframes. Each specification states the route, the audience, the data the screen requests, the principal controls and the states it must be able to render, so that the screen can be checked against the implementation and reproduced as a figure without invention.

Table 4.11: Frontend route inventory

Route	Audience	Component	Data requested	Purpose
`/login`, `/admin/login`, `/field-officer/login`	Public	`Login.jsx`	`POST /api/auth/login/{admin|field-officer}`	Role-specific sign-in; a wrong-role account is refused with a message naming the other form
`/verify`	Public	`Verification.jsx`	`POST /api/verify`	Student-record verification by QR
`/verify-certificate`	Public	`CertificateVerification.jsx`	`POST /api/certificates/verify`	Certificate authenticity
`/dashboard`	Administrator	`AdminDashboard.jsx`	`GET /api/admin/dashboard`	Operational overview
`/duplicate-review`	Administrator	`DuplicateReview.jsx`	`GET /api/duplicate-flags`, `PATCH /api/duplicate-flags/:id`, `GET /api/duplicate-flags/:id/identifiers`, `POST /api/duplicate-flags/scan`	Case adjudication and registry rescan
`/disbursements`	Administrator, Field Officer	`Disbursements.jsx`, `AwardCertificatePanel.jsx`	`/api/awards`, `/api/disbursements`, `/api/disbursements/exceptions`, and `/api/students` **only for field officers**	Awards, payments, exceptions, certificates
`/reports`	Administrator	`Reports.jsx`	`GET /api/reports/export`	Date-range PDF export
`/audit`	Administrator	`AuditLogs.jsx`	`GET /api/audit-logs`	Accountability review
`/users`	Administrator	`UserManagement.jsx`	`GET`, `POST /api/users`	Account provisioning
`/settings`	Administrator	`Settings.jsx`	`GET`, `PUT /api/settings/threshold`	Thresholds
`/field-officer`	Field Officer	`FieldOfficerDashboard.jsx`	`GET /api/auth/me`	Landing page
`/students`	Field Officer	`Students.jsx`	`GET /api/students`	Registry list
`/students/register`	Field Officer	`StudentRegistration.jsx`	`GET /api/students/schools`, `POST /api/students`	Registration and screening feedback
`/eligibility`	Field Officer	`Eligibility.jsx`	`POST /api/eligibility`	Rule-based eligibility assessment

Shared design decisions apply to every screen: navigation is generated from the signed-in role, so a role never sees a link it cannot use; `ProtectedRoute` redirects an unauthenticated visitor to `/login` and a wrong-role visitor to `/dashboard`; and the two public pages use a separate minimal layout with no navigation, because their audience is not a system user.

4.5.1 Sign-in pages

[Insert Figure 4.13: Sign-in screen here]

Figure 4.13: Role-specific sign-in screen. The figure must show the SBMS identity block, an e-mail field, a password field, a submit button with a busy state ("Signing in…"), an inline error region (`role="alert"`) that renders "Invalid email or password" or the wrong-role message, and the link that switches to the other role's form. It must show the state **after** a failed attempt, because the error region is part of the design.

4.5.2 Field-officer workspace: registry list and eligibility

[Insert Figure 4.14: Field-officer student registry list here]

Figure 4.14: Student registry list (field officer). The figure must show the SBMS header, the role navigation for a field officer (Dashboard, Students, Register Student, Eligibility, Disbursements), a tabular list of registered students with name, date of birth, school identifier and registration date, and an empty state ("No students registered yet") alongside the "Register Student" action. It must not show QR tokens, model scores or duplicate-case information, none of which this screen receives.

[Insert Figure 4.15: Eligibility assessment screen here]

Figure 4.15: Eligibility assessment screen. The figure must show the five inputs (fee balance, household income, vulnerability status, siblings in school, prior bursary history), the computed score, the category, the per-factor breakdown and the human-readable reasons list, because the rule engine returns `{score, category, breakdown, reasons}` and the design intent is explainability rather than a single number.

4.5.3 Registration and screening-confirmation screen

[Insert Figure 4.16: Student registration form here]

Figure 4.16: Student registration form. The figure must show the two-column name fields (first and last name, combined by the client into `full_name`), the date-of-birth date picker, the school selector populated from `GET /api/students/schools` with a disabled "Choose school…" placeholder, the national identifier field, the two **optional** identifier fields (birth certificate number, UPI) with their explanatory comment, the phone field with the `+254712345678 or 0712345678` hint, the submit button with its "Registering…" busy state, and the inline error alert. The figure must not show a model selector, a threshold or a score.

[Insert Figure 4.17: Screening confirmation panel here]

Figure 4.17: Registration confirmation with screening outcome. The figure must show the QR code image returned by the API, the student's name, and the screening sentence built by `StudentRegistration.jsx:7-22`. Three states must be illustrated, because the component renders them differently:

State	Text rendered	Visual treatment
No candidate	"Checked against N existing student(s)." with no flag	Neutral
Candidate(s) under the review policy	"Possible duplicate match found. An administrator must review this case before any disbursement." plus "Checked against N existing students; M candidates flagged for a second look; K further matches recorded in the audit log without a separate case."	Amber
Exact identifier agreement	The same sentence plus the exact identifier named in plain language ("UPI", "birth certificate number", "National ID")	Red

The panel must show no model version, no threshold and no score, because the officer's screen is deliberately limited to what the officer can act on.

4.5.4 Administrator dashboard

[Insert Figure 4.18: Administrator dashboard here]

Figure 4.18: Administrator dashboard. The figure must show the counters the endpoint actually returns: total students, eligible students (`eligible` + `highly_eligible`), synthetic students in the registry, and open duplicate cases; a six-month trend of registrations and of disbursements rendered as line or bar charts; the recent activity list (four most recent audit actions); and the accountability block — approved awards, disbursed awards, approved

awards with no payment, disbursed amount, awaiting clearance, awaiting receipt acknowledgement, overdue receipts (or "deadline not configured"), partial awards and amount discrepancies. A "last updated" timestamp must appear, because the payload carries `lastUpdated`.

4.5.5 Duplicate review screen

[Insert Figure 4.19: Duplicate review queue with a case open here]

Figure 4.19: Duplicate review queue. The figure must show the queue with each case's two records side by side (name, date of birth, school identifier, masked identifiers), the priority badge (`high` / `standard`), the model score, the model version (`xgboost:<hash prefix>`), the review threshold and the review reason; the policy line stating both cutoffs, the aggregation policy and "provisional; validation is unavailable"; the reveal-identifiers action with its warning that the reveal is audited; and the decision control requiring a decision, a reason of at least five characters and optional notes. The figure must show the state after a decision has been recorded, including the case history entry.

4.5.6 Awards, payments and certificates

[Insert Figure 4.20: Awards and disbursements screen with the certificate panel here]

Figure 4.20: Awards and disbursements screen. The figure must show the awards table (student, requested amount, approved amount, status, verification status, total disbursed, payment count, open duplicate count, verification history), the payment record form (amount, method, reference, date), the payment status actions (clear with evidence, fail or cancel with a reason), the acknowledgement action (amount received and evidence reference) and the exceptions panel with its reason tags. It must also show the collapsed and expanded **Bursary certificates** panel: issue with an optional award period, the printed certificate view with its QR code, the PDF download action, and the revoke action with a mandatory reason, plus the explicit statement that "approval is not payment".

4.5.7 Administrative support screens: reports, audit, users and settings

[Insert Figure 4.21: Reporting, audit-log, user-management and settings screens here]

Figure 4.21: Administrative support screens. Four panels, because they share one design purpose — making administrative actions visible:

Screen	Controls the figure must show	Data shown
Reports	Start date, end date, "Export PDF" action, and the validation message "Start date cannot be after end date"	The generated PDF as a downloaded document, not on screen
Audit logs	Reverse-chronological table with actor, action, table, record and details, plus a readable rendering of the duplicate-screening counters (`considered · qualifying · suppressed`)	`audit_logs` rows
Users	Account list with name, e-mail, role and active state; create form with name, e-mail, role and password	`users` rows (never the hash)
Settings	Three numeric inputs (review threshold, high-priority threshold, model cut-off) with 0–1 validation	Current values, and the statement that the score is not a calibrated probability

4.5.8 Public verification pages

[Insert Figure 4.22: Public student-record verification page here]

Figure 4.22: Student-record verification page. The figure must show the minimal layout with no navigation, the heading "Student record verification", the sub-line "Scanned SBMS student QR code", and the five states the page renders: checking, record found, not verified, rate limited and connection problem. For "record found" it must show the three explanatory bullets, including "It confirms that the record exists. It does not confirm identity, eligibility, approval or payment." No student data may appear in any state.

[Insert Figure 4.23: Public certificate verification page here]

Figure 4.23: Certificate verification page. The figure must show the four outcome states — verified, revoked, not current, not recognised — and, for a verified certificate, the fields the endpoint actually returns: certificate reference (`SBMS-CERT-<year>-<8 hex>`), award period, issue date, approval date, award status and issuing authority ("Student Beneficiary Management System (SBMS)"). It must **not** show the student's name, UPI, school or the approved amount, because the public response omits them.

4.6 Database schema

4.6.1 Schema organisation and migration policy

The schema is a baseline file plus an ordered set of migrations. The baseline (`backend/src/db/schema.sql`) creates `users`, `students`, `duplicate_flags`, `duplicate_review_events`, `disbursements`, `audit_logs`, `reports` and the legacy `beneficiaries` table, and is written to be idempotent (`CREATE TABLE IF NOT EXISTS`, `ADD COLUMN IF NOT EXISTS`, constraint guards inside `DO $$` blocks). The migrations add what was built later.

Table 4.12: Migration inventory

File	Purpose	Status
`20261019_admin_field_officer_roles.sql`	Two-role model and the role `CHECK` constraint	Applied to the test database
`20261019_duplicate_review_workflow.sql`	Review events and case-status semantics	Applied
`20261019_enforce_supported_schools.sql`	Restricts `school_code` to the approved list	Applied
`20261019_field_officer_registration.sql`	Field-officer account support	Applied
`20261019_limit_synthetic_schools.sql`	Keeps synthetic rows within the school list	Applied
`20261019_remove_duplicate_case_assignment.sql`	Removes an earlier assignment concept	Applied
`20261019_standardize_student_records.sql`	Normalises stored student fields	Applied
`20261019_synthetic_student_marker.sql`	Adds `is_synthetic`	Applied
`20261022_ml_duplicate_provenance.sql` (+ `.down.sql`)	Adds `model_version`, `review_threshold`, `comparison_features` and the unordered-pair unique index, with a pre-flight refusing to collapse existing duplicate pairs	Applied
`20261023_bursary_award_reconciliation.sql`	Adds `bursary_awards`, `bursary_award_verification_events`, the payment and receipt columns with constraints, and `disbursement_reconciliation_events`	Applied
`20261024_add_user_activation_flag.sql`	Adds `users.is_active`	Applied
`20261025_legacy_beneficiaries_baseline.sql`	Formalises the legacy `beneficiaries` table	Applied
`20261031_bursary_certificates.sql`	Adds `bursary_certificates`	Applied to the disposable test database; required by readiness
`20261023_retain_primary_accounts.sql`	Manual account-retention script	**Not part of the automatic migration order**

Migrations are not applied by the application. `backend/scripts/migrate.js` applies them in filename order and records them in `sbms_schema_migrations`; the operator is expected to back up and verify a restore first (`README.md`; `docs/database-recovery-rehearsal-2026-10-31.md`). The recorded state of the disposable test database after the last

full run was **12 base tables, 118 columns, 56 constraints, 21 indexes, 2 extensions and 12 applied migrations** (`docs/FINAL_TEST_REPORT.md` §5).

4.6.2 Column-level design of the core tables

Table 4.13: `users`

Column	Type	Constraints	Notes
`id`	`UUID`	Primary key, `gen_random_uuid()`	Non-sequential, so identifiers cannot be enumerated
`full_name`	`VARCHAR(255)`	Not null	
`email`	`VARCHAR(255)`	Not null, unique	Lower-cased by the API before storage
`password_hash`	`TEXT`	Not null	bcrypt, cost 12; never returned by any endpoint
`role`	`VARCHAR(50)`	Not null, default `field_officer`, `CHECK role IN ('admin','field_officer')`	Enforced twice, in the table and in a guarded `ALTER`
`is_active`	`BOOLEAN`	Not null, default `TRUE`	Authentication requires `TRUE`
`created_at`	`TIMESTAMP`	Default `NOW()`	

Table 4.14: `students`

Column	Type	Constraints	Notes
`id`	`UUID`	Primary key, `gen_random_uuid()`	
`full_name`	`VARCHAR(255)`	Not null	Whitespace-normalised by the API
`date_of_birth`	`DATE`	Nullable	Compared as a calendar date at the predictor boundary
`school_code`	`VARCHAR(100)`	Nullable, `CHECK` against the five approved identifiers	Holds a school *name* in practice; no authoritative code list exists
`guardian_id`	`VARCHAR(100)`	Nullable	Model feature
`upi`	`VARCHAR(100)`	Nullable; earlier `UNIQUE` constraint deliberately dropped	Model feature; exact agreement is also a review signal
`birth_certificate_number`	`VARCHAR(100)`	Nullable	Added by an idempotent block that also renames the earlier `birth_certificate_entry_number`
`national_id_number`	`VARCHAR(100)`	Nullable	Review signal, **not** a model feature
`primary_phone`	`VARCHAR(50)`	Nullable	Normalised to `+254` form
`qr_token`	`VARCHAR(100)`	Nullable, **partial unique index** where not null	64 lowercase hex characters; never returned by the API
`is_synthetic`	`BOOLEAN`	Not null, default `FALSE`	Distinguishes demonstration rows on the dashboard
`eligibility_score` / `eligibility_category` / `eligibility_assessed_at`	`INTEGER` / `VARCHAR(50)` / `TIMESTAMP`	Nullable	0–100 and one of four categories
`created_at`	`TIMESTAMP`	Default `NOW()`	

Table 4.15: `duplicate_flags` and `duplicate_review_events`

Table / column	Type	Constraints
`duplicate_flags.id`	`UUID`	Primary key
`student_id`, `matched_student_id`	`UUID`	Not null, `REFERENCES students(id) ON DELETE CASCADE`
`probability`	`NUMERIC(5,4)`	Not null; comment records that model rows store an uncalibrated score

Table / column	Type	Constraints
`status`	`VARCHAR(50)`	Not null, default `pending`
`model_version`	`TEXT`	Nullable; `xgboost:<sha256>` for model rows
`review_threshold`	`NUMERIC(5,4)`	Nullable; the cutoff in force when the case was created
`comparison_features`	`JSONB`	Nullable; the six-feature vector
`resolved_by`, `resolved_at`, `updated_at`, `created_at`	`UUID` / `TIMESTAMP`	`resolved_by REFERENCES users(id)`; `updated_at` not null
(index)	unique	`LEAST(student_id, matched_student_id), GREATEST(student_id, matched_student_id)`
`duplicate_review_events.id`	`UUID`	Primary key
`duplicate_flag_id`	`UUID`	Not null, `REFERENCES duplicate_flags(id) ON DELETE CASCADE`
`reviewer_id`	`UUID`	Not null, `REFERENCES users(id)`
`decision`	`VARCHAR(30)`	Not null, `CHECK IN ('confirmed_duplicate','not_duplicate','needs_verification')`
`reason`	`TEXT`	Not null, `CHECK length(btrim(reason)) > 0`
`notes`	`TEXT`	Nullable
(index)	index	`(duplicate_flag_id, created_at DESC)`

Table 4.16: `bursary_awards` and `bursary_award_verification_events` (migration `20261023`)

Table / column	Type	Constraints
`bursary_awards.id`	`UUID`	Primary key
`student_id`	`UUID`	Not null, `REFERENCES students(id) ON DELETE RESTRICT`
`school_code`	`VARCHAR(100)`	Not null (copied from the student at submission)
`requested_amount`, `approved_amount`	`NUMERIC(12,2)`	`requested_amount > 0`; `approved_amount` null or `> 0`
`status`	`VARCHAR(20)`	`CHECK IN ('submitted','approved','rejected')`
`verification_status`	`VARCHAR(30)`	`CHECK IN ('not_checked','verified','not_found','needs_investigation')`
`verification_notes`, `verification_evidence_ref`	`TEXT` / `VARCHAR(500)`	Evidence mandatory for `verified`
`verified_by`, `verified_at`, `decided_by`, `decided_at`, `submitted_by`	`UUID` / `TIMESTAMPTZ`	`REFERENCES users(id) ON DELETE RESTRICT`
`decision_reason`	`TEXT`	≥ 5 characters when a decision exists
(constraints)	`CHECK`	`bursary_awards_decision_fields_check`, `bursary_awards_approval_verification_check`, `bursary_awards_verification_fields_check`
(indexes)	index	`(status, created_at DESC)`, `(student_id, created_at DESC)`
`bursary_award_verification_events.*`	—	`outcome`, `notes` (≥ 5 characters), `evidence_ref` (mandatory for `verified`), `actor_id`, `created_at`; index `(award_id, created_at DESC)`

Table 4.17: `disbursements` and `disbursement_reconciliation_events`

Column	Type	Constraints / notes
`id`, `student_id`, `amount`, `disbursement_date`, `status`, `created_by`, `created_at`	existing	`amount NUMERIC(12,2) NOT NULL`; `status VARCHAR(50) DEFAULT 'completed'`
`award_id`	`UUID`	Nullable, `REFERENCES bursary_awards(id) ON DELETE RESTRICT`; null for legacy payments
`payment_method`, `payment_reference`	`VARCHAR`	Nullable
`payment_status`	`VARCHAR(20)`	`CHECK IN ('issued','cleared','failed','cancelled')`
`cleared_by`, `cleared_at`, `clearance_evidence_ref`	—	Required together when `payment_status = 'cleared'`
`receipt_status`	`VARCHAR(30)`	`CHECK IN ('awaiting_acknowledgement','acknowledged','discrepancy')`
`acknowledged_amount`, `acknowledged_by`, `acknowledged_at`, `acknowledgement_method`, `acknowledgement_evidence_ref`, `acknowledgement_notes`	—	Required together when a receipt status is recorded; `acknowledgement_method` must be `officer_assisted`
(index)	index	`(award_id, payment_status, receipt_status)`
`disbursement_reconciliation_events`	—	`event_type CHECK IN ('payment_issued','payment_cleared','payment_failed','payment_cancelled','payment_linked_to_award','receipt_acknowledged')`; index `(disbursement_id, created_at DESC)`

Table 4.18: `bursary_certificates` (migration `20261031`)

Column	Type	Constraints
`id`	`UUID`	Primary key
`award_id`	`UUID`	Not null, **`UNIQUE`**, `REFERENCES bursary_awards(id) ON DELETE RESTRICT`
`certificate_reference`	`VARCHAR(40)`	Not null, unique; `SBMS-CERT-<year>-<8 uppercase hex>`
`verification_token`	`VARCHAR(100)`	Not null, unique, `CHECK ~ '^[a-f0-9]{64}$'`
`award_period`	`VARCHAR(20)`	Not null, `CHECK ~ '^[0-9]{4}(/[0-9]{4})?$'`
`student_name`, `student_upi`, `school_code`	—	Snapshot of the facts printed on the document
`approved_amount`	`NUMERIC(12,2)`	Not null, `> 0`
`award_approved_at`	`TIMESTAMPTZ`	Not null
`status`	`VARCHAR(20)`	`CHECK IN ('active','revoked')`
`issued_by`, `issued_at`	`UUID` / `TIMESTAMPTZ`	Not null
`revoked_by`, `revoked_at`, `revocation_reason`	—	`bursary_certificates_revocation_check` requires all three together with a reason of ≥ 5 characters, and all three null while `active`; `bursary_certificates_period_check` and `bursary_certificates_token_check` guard the formats
(index)	index	`(status, issued_at DESC)`

Table 4.19: `audit_logs`, `reports` and the legacy `beneficiaries`

Table	Columns	Notes
`audit_logs`	`id`, `user_id`, `action VARCHAR(100) NOT NULL`, `table_name`, `record_id UUID`, `details JSONB`, `created_at`	Append-only by convention; no trigger, no immutability constraint, no retention policy
`reports`	`id`, `generated_by`, `report_type`, `start_date`, `end_date`, `file_path`, `created_at`	Metadata for an exported PDF; the file itself is written to `reports/` on the API host
`beneficiaries`	`id SERIAL`, `name`, `school`, `grade_level`, `scholarship_amount NUMERIC DEFAULT 0`, `created_at`	Legacy shape reproduced deliberately: serial key, **no foreign keys**, unresolved `scholarship_amount` semantics

4.6.3 Integrity, indexing and normalisation notes

- **Normalisation level.** The schema is in third normal form for its operational data: student identifiers are attributes of `students`, award decisions are attributes of `bursary_awards`, and no non-key attribute depends on another non-key attribute. The two deliberate exceptions are the *snapshot* columns on `bursary_certificates` (which duplicate award facts to make a printed document immutable) and the *derived* counters computed at read time rather than stored, so that no cached aggregate can drift from its source.
- **Referential actions.** Review events and audit-adjacent children cascade from their parent case (`ON DELETE CASCADE`), because a review history has no meaning without its case. Awards, payments and certificates use `ON DELETE RESTRICT`, because financial and documentary records must not be removable through a parent deletion.
- **Indexes.** Seven indexes are material to query performance and constraint enforcement: the partial unique index on `students.qr_token`, the unordered-pair unique index on `duplicate_flags`, the two review-event and award indexes, the reconciliation-event index, the payment-status composite index and the certificate status index.
- **What the schema cannot enforce.** Nothing prevents a `pending` case from existing for two records that are in fact the same person: that judgement belongs to the reviewer. Nothing records *who* created a student row; the creating user is only in `audit_logs`. The legacy `beneficiaries` table remains unreconciled.

4.7 System architecture (physical deployment)

The logical architecture in §4.4.1 was realised in two deployment configurations: a single-host development configuration and a managed cloud configuration on Microsoft Azure that was provisioned and verified live on 31 October 2026 (`docs/azure-deployment-log-2026-10-31.md` §1–§2).

[Insert Figure 4.24: Physical deployment architecture on Azure here]

Figure 4.24: Physical deployment architecture. The figure must show:

Element	Configuration	Evidence
API container	Azure Container Apps app `ca-sbms-api` in environment `cae-sbms`, region UAE North, `minReplicas 0`, `maxReplicas 1`, environment created with `--logs-destination none` (no log-analytics workspace)	`docs/azure-deployment-log-2026-10-31.md` §1, §3 (D5, D7)
API image	`ghcr.io/eggxellent/sbms-api`, built by the dispatchable workflow `publish-api-image.yml` from the repository `Dockerfile`, `linux/amd64`, 1.37 GB after cleanup (was 2.22 GB)	`.github/workflows/publish-api-image.yml`; deployment log §1
Image gates	The workflow refuses an image whose predictor fails a known-duplicate/known-unrelated scoring assertion, whose feature order differs, that contains any `nvidia-*` package, that contains a `.env*` file, or that contains a `deploy/` directory	workflow gate steps
Frontend	Azure App Service `sbms-web` on the F1 (free) Linux tier, static build published as a zip, SPA rewrite so client routes resolve	`render.yaml` documents an equivalent Render topology; deployment log §1, §7
Database	PostgreSQL Flexible Server `pg-sbms` (B1MS, 32 GiB, PostgreSQL 16), database `sbms`, public access with TLS, stopped between demonstrations	deployment log §1, §3 (D4, D8)
Secrets	`DATABASE_URL`, `JWT_SECRET` and bootstrap credentials as container-app secrets; account passwords only in the git-ignored `deploy/azure/.secrets.json`	deployment log §1, §5

Element	Configuration	Evidence
Networking and TLS	HTTPS only; API FQDN `ca-sbms-api.ambitiousstone-f2de8f67.uaenorth.azurecontainerapps.io`; frontend `sbms-web.azurewebsites.net`; CORS allowlist and `PUBLIC_APP_URL` both set to the frontend origin	`render.yaml` env wiring; deployment log §2
Health	`GET /api/health` as container health probe; `GET /api/ready` verifies expected database identity and required tables	`index.js:69-86`; `services/databaseReadiness.js`

Three deployment facts matter for Chapter Five and must be stated plainly. First, the cloud deployment used **one container for both the API and the predictor**, because the predictor is a child process of the API; the figure must not draw them as two services. Second, the frontend and API are **separate origins**, which is why the cross-origin wiring (`VITE_API_ORIGIN`, `CORS_ORIGINS`, `PUBLIC_APP_URL`) is part of the architecture rather than an afterthought — a misconfiguration here produced a real defect, recorded in Chapter Five §5.6.3. Third, the database is **stopped** outside demonstration windows to keep the project inside a student credit budget, so "the deployment is live" is not the same claim as "the system is continuously available".

Table 4.20: Development and test environment topology

Environment	Components	Purpose
Local development	API on port 5000 (`nodemon`), Vite dev server on port 4174 proxying `/api`, local PostgreSQL, `ml-service/.venv` predictor	Development and manual checks
Disposable test database	`sbms_test` on loopback, role `sbms_test_runner`, guarded by `backend/tests/testDatabaseGuard.js`, which fails closed unless `NODE_ENV=test` and both `DATABASE_URL` and `SBMS_TEST_DATABASE_URL` name that same loopback database	Automated Jest suites
Research environment	Python virtual environment with `pandas`, `scikit-learn`, `xgboost`, `jellyfish`; artefacts under `ml-service/experiments/` and `ml-service/evaluation/`	Training, evaluation and the separate research-only models

4.8 Chapter summary

This chapter converted the project's intentions into a specification of the delivered system. It established thirty-seven functional requirements (thirty delivered, three recorded as proposal requirements with documented deviation or exclusion, and four recorded as unsatisfied gaps) and twenty-four non-functional requirements, of which three are recorded as not met; it stated the data requirements, including the six-feature contract, the normalisation rules and the absence of any real beneficiary data; and it defined five interfaces with their protocols, authentication and error behaviour.

It then presented the analysis models — context, use case, activity, class, system sequence, sequence, entity-relationship and ML-pipeline diagrams — as specifications that can be drawn without inventing behaviour, and the design of the system: a three-tier architecture in which the machine-learning component is a supervised child process of the API; a module structure with clear layering and two acknowledged weaknesses (a dead legacy rule engine and an unreconciled legacy table); a two-role access matrix whose most-contested rule (`/api/students` is field-officer-only) is shown to be a designed policy rather than a defect; a duplicate-detection design that separates scoring from case selection; four explicit state machines; a security design mapped threat by threat to its control; a relational schema described column by column with its constraints, indexes and deliberate exceptions; and a physical deployment architecture that was actually provisioned.

Two conclusions follow from the analysis and are carried into Chapter Five. The first is that the system's integrity depends less on the classifier than on the two human gates that stand between a possible duplicate and a payment; those gates, not the model, are what the design offers against the problem stated in Chapter One. The second is that the design's principal technical weakness is candidate generation: comparing every new record against the entire registry makes the *volume* of review work grow with the registry, and no threshold setting repairs that —

which is why Chapter Five evaluates the model and the aggregation policy separately, and why Chapter Six proposes blocking as future work.

Chapter Five now reports how this design was implemented, how it was tested, what the evidence establishes, and what it does not.